Tutorial Didático de Python

PARTE III - LISTAS, TUPLAS E TRATAMENTO DE EXCEÇÕES



RAFAEL R. PACHECO

SUMÁRIO

Introdução.....1

CAPÍTULO 1 – Estruturas de Dados.....2

1.1 Definição e Principais Estruturas.....2

1.2 Principais Características.....5

CAPÍTULO 2 – Listas.....6

2.1 Definição e Sintaxe.....6

2.2 Acessando Elementos – Índices das Listas.....8

2.3 Modificando Elementos das Listas.....10

2.4 Adicionando Elementos.....17

2.4.1 Método append().....17

2.4.2 Método insert().....18

2.3.4 Método extend().....20

2.5 Removendo Elementos.....23

2.5.1 Método remove().....23

2.5.2 Método pop().....25

2.5.3 Comando del.....28

2.5.4 Método clear().....30

2.6 Métodos Adicionais.....31

2.6.1 Método sort().....31

2.6.2 Método reverse().....33

2.6.3 Método count().....34

2.6.4 Método index().....34

2.7 Copiando Listas.....36

2.7.1 Como NÃO Copiar Listas.....36

2.7.2 Método copy().....38

2.7.3 Cópia por Slicing.....39

2.8 Funções Úteis – len(), sum(), min(), max(), enumerate().....40

2.8.1 Função len().....40

2.8.2 Função sum().....41

2.8.3 Função min().....42

2.8.4 Função max().....43

2.8.5 Função enumerate().....44

2.9 Listas Aninhadas.....45

2.9.1 Acessando Elementos.....47

2.9.2 Modificando Elementos.....49

2.9.3 Representação de Matrizes.....52

2.9.4 Iteração de Listas Aninhadas.....54

2.10 List Comprehension.....55

CAPÍTULO 3 – Tuplas.....58

3.1 Definição e Sintaxe.....58

3.2 Acessando Elementos.....61

3.3 Imutabilidade……...61

3.4 Métodos de Tuplas.....63

3.5 Funções Úteis.....64

3.6 Tuplas Aninhadas.....65

3.7 Quando Utilizar Tuplas?.....65

CAPÍTULO 4 – Tratamento de Exceções.....67

4.1 Exceções.....67

4.2 Tratamento de Exceções.....68

4.3 Estrutura try – except.....70

4.3.1 Tratando Exceções Específicas.....75

4.3.2 Tratando Múltiplas Exceções.....77

4.3.3 Múltiplas Exceções No Mesmo except.....80

4.4 Cláusula else.....82

4.5 Cláusula finally.....84

4.6 Acessando a Exceção com as.....87

4.7 Lançando Exceções – raise.....89

4.8 Considerações Finais.....91

Introdução

Este tutorial foi desenvolvido como projeto de extensão no curso de Ciência da Computação da UNIP, sendo, portanto, totalmente gratuito. O objetivo desse tutorial é ensinar a linguagem de programação Python de forma simples e didática, permitindo que qualquer pessoa possa aprender a programar.

Assim como a primeira e a segunda partes, também temos por objetivo manter uma linguagem simples e acessível, com foco na didática e nos exemplos para que o material sirva como apoio aos estudos universitários e outros cursos de Python.

Esta terceira parte continua a exposição dos conceitos básicos de Python. Estudaremos as estruturas de dados, em especial as listas e as tuplas. Além disso, iremos introduzir conceitos de tratamento de exceções.

Bons estudos!

CAPÍTULO 1 – Estruturas de Dados

1.1 Definição e Principais Estruturas

As estruturas de dados são formas especializadas de armazenar, organizar e manipular dados. Vamos pegar as strings como exemplo, pois já estudamos na parte 2 do tutorial, e comparar com uma variável de tipo int para facilitar a compreensão.

Uma string pode conter vários caracteres armazenados em uma mesma variável. Podemos criar uma string contendo letras do alfabeto, números, espaços em branco, pontuação e armazenar tudo isso na memória sob uma única variável. **Porém, todos esses dados serão caracteres e não tipos como int, float ou bool!**

Já o int somente admite um número por variável. Não é possível que uma variável do tipo int armazene vários números inteiros ao mesmo tempo.

Vamos relembrar como funciona a dinâmica entre a variáveis e a memória do computador! Quando criamos uma variável e atribuímos um conteúdo (dados) a ela, o computador precisa salvar isso na memória para poder acessar e trabalhar com essa informação.

Veja o exemplo:

```
1    numero = 5
2
```

Quando criamos essa variável chamada “numero” e atribuímos 5 como valor, o computador procurou um espaço vazio na memória e salvou a informação de que a variável de nome “numero” está guardando o valor inteiro 5.

Se usarmos o comando print(numero), o computador vai procurar na memória onde está salva essa variável chamada “numero” e vai acessar o valor dela, imprimindo 5 no console.

Observe que o tipo int somente consegue armazenar um número inteiro por vez. Pode até ser um número muito grande, mas não tem como armazenar mais de um número ao mesmo tempo.

Mas e se nós precisássemos que uma variável armazenasse mais de um número? Pode ser que seja conveniente para um programa nosso que tenhamos uma variável que guarde vários números, de forma que, ao pedir que o computador acesse o conteúdo dessa variável, ele nos entregue todos esses números, em vez de apenas um.

É justamente para resolver essa necessidade que as estruturas de dados existem. Uma mesma variável pode representar vários valores diferentes na memória (ou seja, pode armazenar mais de um valor de uma vez). Vamos usar uma string como exemplo, pois já estudamos bem o seu funcionamento na parte 2.

Considere a seguinte string:

```
1    frase = "João comprou 13 laranjas e pagou R$1,30 por cada!"
2
```

Quando criamos e inicializamos essa string, o computador salva o seu conteúdo em memória. No momento em que essa string for chamada - por exemplo damos um print(frase) – o computador acessa o conteúdo da string e irá exibir todos esses caracteres que nós armazenamos (cada letra, número, espaço vazio e pontuação é um caractere. A string permite que uma só variável armazene diversos caracteres).

Então nesse exemplo conseguimos que uma só variável tenha armazenado vários caracteres.

Acontece que as strings são apenas um tipo de estrutura de dados que temos à disposição e não resolvem todos os nossos problemas. Vamos supor que temos 5 strings diferentes, cada uma representando um aluno:

```
1    aluno_1 = "João"
2    aluno_2 = "Maria"
3    aluno_3 = "José"
4    aluno_4 = "Marta"
5    aluno_5 = "Pedro"
6
```

E se for conveniente no nosso programa criar uma estrutura que contenha todos os 5 alunos listados? Nós poderíamos criar uma nova string chamada “alunos” e colocar todos eles separados por vírgulas ou espaços, por exemplo. **Mas nem sempre isso será a melhor solução!**

Nessa hipótese não estaríamos criando uma estrutura com diversas strings, mas somente uma string nova. Como veremos em breve, existem poderosas estruturas que podem nos auxiliar a armazenar diversos dados, muitas vezes até mesmo de tipos diferentes, de forma extremamente fácil e eficiente.

Por enquanto a lição é: sempre que pensarmos em estruturas de dados temos de lembrar que isso é uma forma de armazenar vários valores em uma variável.

Nós vamos estudar as principais ferramentas que o Python nos oferece para atender as nossas necessidades relativas a essa questão. Nesse tutorial nós iremos estudar as principais estruturas de dados que o Python nos oferece. São as seguintes:

- Strings (já estudadas);

- Listas;

- Tuplas;

- Conjuntos (estudaremos na próxima parte);

- Dicionários (estudaremos na próxima parte).

1.2 Principais Caracterísiticas

As estruturas de dados possuem características próprias que permitem escolhermos qual a estrutura usar para melhor atender à nossa necessidade atual no programa.

Iterabilidade - As estruturas de dados que estudaremos são iteráveis, ou seja, podem ser percorridas por um laço for. Na parte 2 do tutorial estudamos a fundo esse tipo de laço e vimos a iteração ocorrendo na prática com as strings.

Mutabilidade - Algumas estruturas são mutáveis, ou seja, podem sofrer alterações após a sua criação. Outras, por outro lado, são imutáveis. Uma vez criadas elas não podem mais ser modificadas.

Organização - Por fim, também podemos classificar as estruturas entre ordenadas e não ordenadas. As ordenadas mantêm uma sequência definida de elementos. Por outro lado, as não ordenadas não possuem uma ordem definida, e a posição dos elementos não é relevante para elas.

Analisaremos cada tipo de estrutura de dados (com exceção das strings, que já estudamos a fundo na parte 2 do tutorial, e dos dicionários e dos conjuntos, que estudaremos na próxima parte) e explicaremos melhor essas características.

CAPÍTULO 2 – Listas

2.1 Definição e Sintaxe

As listas são estruturas de dados que armazenam diversos valores em sequência e em uma única variável. Esses valores podem ser números, texto, valores booleanos, variáveis e até mesmo outras estruturas de dados.

Isso significa que as listas representam um tipo extremamente versátil no Python, que tem a capacidade de armazenar diversos elementos de tipos diferentes ao mesmo tempo. Essa estrutura é iterável, mutável e ordenada.

Devido a essas características, as listas são a estrutura de dados mais utilizada no Python e oferecem muitas soluções para as necessidades dos programadores no seu dia a dia.

As listas, portanto, podem ter vários elementos e todos eles serão separados por vírgula. A principal sintaxe para criar uma lista é:

nome_da_variável = [item1, item2, item3]

Então, para criar uma lista nós declaramos uma variável que irá receber elementos separados por vírgula e dentro de colchetes. Vamos criar uma lista chamada numeros e com os números 1, 2 e 3:

```
1    numeros = [1, 2, 3]
2
```

Essa nossa lista tem apenas números inteiros nela, mas também podemos criar uma lista com outros tipos diferentes misturados. Para isso vamos criar uma lista chamada exemplo e ela conterá um int, um float, uma string e a lista numeros que acabamos de criar!

```
numeros = [1, 2, 3]

exemplo = [5, 3.14, "Curso de Python", numeros]

```

Veja que a lista realmente suporta vários valores de tipos diferentes ao mesmo tempo. Temos até uma lista dentro de outra lista e isso não quebrou o programa! (“numeros” é uma lista e está dentro da nossa nova lista exemplo).

Além dessa forma principal e mais utilizada de criar uma lista, ainda temos outras duas possibilidades. Vamos analisar cada uma delas com calma.

Nós podemos criar uma lista vazia usando essa mesma sintaxe, mas não vamos colocar nenhum elemento dentro dela logo na criação.

```
vazia = []

```

Com isso criamos uma lista que não tem nada dentro. Como as listas são mutáveis, podemos colocar os itens dentro dela depois, conforme for conveniente para o nosso programa. Não se preocupe! Em breve vamos mostrar como isso funciona.

A outra forma é usando list(), que é uma função do Python para criar listas.

```
minha_lista_vazia = list() #cria uma lista vazia
minha_lista = list("iterável") # cria uma lista a partir dos elementos de um iterável

```

No primeiro caso, criamos uma lista vazia assim. Já no segundo, cada letra da string “iterável” irá virar um elemento da lista. Vamos dar print em ambas para ver isso na prática:

```
minha_lista_vazia = list() #cria uma lista vazia
minha_lista = list("iterável") # cria uma lista a partir dos elementos de um iterável

print(minha_lista_vazia)
print(minha_lista)

```

```
[]
['i', 't', 'e', 'r', 'á', 'v', 'e', 'l']
```

O primeiro print exibiu uma lista sem nada. O segundo mostrou que a string foi dividida em todos os seus caracteres e cada caractere virou um elemento da lista.

ATENÇÃO: O formato list(iterável) nada mais é do que uma conversão de tipo (casting), que já estudamos na parte 1 do tutorial. Quando usamos list(variável) estamos tentando converter a variável para o tipo lista, mas temos que sempre lembrar que essa conversão somente é possível para objetos iteráveis!

Essas são as principais formas de se criar uma lista, mas, na maior parte das vezes, o mais comum será já criarmos uma lista com os elementos que vamos precisar armazenar ou criar uma lista vazia a ser preenchida depois.

ATENÇÃO: Conforme mencionado, as listas são estruturas de dados ordenadas, ou seja, a ordem de seus elementos é importante. **Uma lista [1, 2, 3] É DIFERENTE da lista [3, 2, 1], ainda que elas possuam os mesmos elementos.**

2.2 Acessando Elementos - Índices das Listas

As listas possuem vários elementos. Esses elementos são organizados em índices que representam suas posições na lista, sendo que o primeiro elemento está no índice 0. Por outro lado, podemos acessar a lista de trás para frente, começando pelo -1. Nós já estudamos esse comportamento com as strings na parte 2 do tutorial. As listas seguem a mesma lógica para acessar os seus elementos.

Como já estudamos a fundo como acessar os elementos das strings e como as listas seguem exatamente a mesma lógica, vamos apenas fazer uma rápida revisão.

Considere a seguinte lista:

numeros = [10, 100, 1000, 10000]

Na posição 0 temos o valor inteiro 10. Na posição 2 temos o valor 1000. Já na posição -1 temos 10000 e na posição -2 temos 1000 novamente (mesmo elemento acessado de outra forma: de trás para frente).

Vamos supor que o programador precise exibir no console o número 1000 (que é o terceiro número da lista, ou seja, índice 2). Ele executará o seguinte comando:

```
1    numeros = [10, 100, 1000, 10000]
2    print(numeros[2])
3
```

Isso irá exibir apenas o valor que ocupa o índice 2 na lista. Lembrando que começamos a contar do 0. É exatamente o que estudamos com as strings.

Caso o programador tente acessar um índice que não exista, o programa apresenta erro e para de funcionar:

```
1    numeros = [10, 100, 1000, 10000]
2    print(numeros[7])
3
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    print(numeros[7])
          ~~~~~~~^^^
IndexError: list index out of range
```

Erro na linha 2. Erro de índice. O índice da lista está fora do intervalo (tamanho) dela. Ou seja, a nossa lista somente possui 4 elementos (ela vai do 0 ao 3) e tentamos acessar o elemento no índice 7, que não existe.

Nas listas também é possível fazer o fatiamento (slicing) e exibir um intervalo de valores. Vamos considerar uma lista de supermercado como exemplo:

```
compras = ["abacaxi", "peito de frango", "suco de laranja", "sabonete",
           "ovos", "papel higiênico", "leite", "abacate", "pasta de dente"]

```

Aqui temos 9 elementos na nossa lista, que são as coisas que precisamos comprar no supermercado. Se quisermos exibir somente os elementos que estejam no intervalo entre 3 e 6, podemos usar a técnica do slicing:

```
compras = ["abacaxi", "peito de frango", "suco de laranja", "sabonete",
           "ovos", "papel higiênico", "leite", "abacate", "pasta de dente"]
print(compras[3:6])

```

Resultado:

```
['sabonete', 'ovos', 'papel higiênico']
```

ATENÇÃO: Lembrando que no slicing o último valor do intervalo não é incluído! Então se o programador tivesse executado print(compras[3:5]), somente “sabonete” e “ovos” teriam sido exibidos, pois slicing de 3 a 5 significa o que está entre 3 (incluído) a 5 (não incluído), ou seja, 3 e 4! Por outro lado, caso o programador desejasse exibir nesse intervalo o elemento de índice 6 da lista, deveria usar print(compras[3:7]).

Tudo o que estudamos sobre slicing nas strings também é aplicável às listas. Para não sermos repetitivos e manter essa revisão breve, sugerimos que o leitor volte na parte 2 do tutorial para relembrar esses conceitos, caso haja dúvidas ou dificuldades.

2.3 Modificando Elementos das Listas

As listas são estruturas de dados mutáveis. Isso significa que podemos modificá-las conforme necessário. Além de acessar os elementos das listas, o programador consegue alterá-los. É possível:

a) Adicionar elementos novos, aumentando o tamanho da lista;
b) Excluir elementos existentes, reduzindo o tamanho da lista;
c) Alterar um elemento, sem modificar o número de elementos da lista.

Vamos começar pela alteração de um elemento já existente. Nesse caso apenas desejamos alterar um elemento da lista, sem mexer nos demais e nem alterar o tamanho da nossa estrutura de dados.

Podemos usar a nossa lista de compras como exemplo. Ela tem 9 elementos que são os itens que precisamos comprar no supermercado. Vamos supor que vimos que já temos bastante papel higiênico em casa, mas que o nosso shampoo está acabando.

Nesse caso, não precisamos comprar mais papel higiênico, mas precisamos comprar shampoo. Então vamos alterar nossa lista para colocar o shampoo no lugar do papel higiênico. A alteração de elementos da lista é muito fácil. Basta indicar o índice do elemento que queremos substituir e atribuir o valor novo! Vejamos:

```
compras = ["abacaxi", "peito de frango", "suco de laranja", "sabonete",
           "ovos", "papel higiênico", "leite", "abacate", "pasta de dente"]

compras[5] = "shampoo"

```

Ou seja: o que estiver no índice 5 da lista de compras agora recebe a string “shampoo”. Antes compras[5] continha a string “papel higiênico” e agora vai passar a ser a string “shampoo”. Vamos usar uns prints para exibir o antes e depois pra ficar bem claro:

```
compras = ["abacaxi", "peito de frango", "suco de laranja", "sabonete",
           "ovos", "papel higiênico", "leite", "abacate", "pasta de dente"]
print("ANTES DA ALTERAÇÃO")
print(compras, "\n")

compras[5] = "shampoo"

print("DEPOIS DA ALTERAÇÃO")
print(compras)

```

Console:

```
ANTES DA ALTERAÇÃO
['abacaxi', 'peito de frango', 'suco de laranja', 'sabonete', 'ovos', 'papel higiênico', 'leite', 'abacate', 'pasta de dente']

DEPOIS DA ALTERAÇÃO
['abacaxi', 'peito de frango', 'suco de laranja', 'sabonete', 'ovos', 'shampoo', 'leite', 'abacate', 'pasta de dente']

Process finished with exit code 0
```

Nosso programa fez o seguinte: criamos a lista de strings compras contendo as coisas que precisamos comprar. Demos um print na lista (e pulamos uma linha extra usando \n para ficar melhor a visualização. Estudamos isso na parte 1). Alteramos o que estava no índice 5 (papel higiênico) para shampoo. Printamos a lista com a alteração.

Essa é a forma mais comum de modificarmos a lista. É simplesmente uma operação de atribuição, igual sempre fazemos com as demais variáveis, mas com a particularidade de termos de indicar o índice que será modificado.

Outra forma possível, mas menos usada, é modificar partes da lista de uma vez só usando slicing (fatiamento). Nesse caso o programador indica qual intervalo ele quer substituir e atribuí novos valores para os elementos que estão nesse intervalo. Esses valores devem ser passados no formato de lista (como se fossem uma mini lista).

Vamos mostrar um exemplo. Queremos mudar “abacaxi”, “peito de frango” e “suco de laranja” por “refrigerante”, “pizza” e “sorvete”. Utilizando o slicing:

```
1   compras = ["abacaxi", "peito de frango", "suco de laranja", "sabonete",
2              "ovos", "papel higiênico", "leite", "abacate", "pasta de dente"]
3
4   compras[0:3] = "refrigerante", "pizza", "sorvete"
```

Printando compras depois de alterar:

```
['refrigerante', 'pizza', 'sorvete', 'sabonete', 'ovos', 'papel higiênico', 'leite', 'abacate', 'pasta de dente']

Process finished with exit code 0
```

Modificamos os 3 primeiros elementos da lista usando slicing. Essa forma de modificar é possível, mas não é a mais usual.

ATENÇÃO: Devemos ter alguns cuidados com a alteração por slicing. Primeiro: não podemos esquecer que o último elemento do intervalo não é inclusivo. Segundo: geralmente é mais conveniente passar o mesmo número de elementos novos que o número de elementos substituídos para evitar comportamento indesejado. Por fim: esses valores que passamos para substituir devem ser ou estarem contidos num iterável.

Esses pontos de atenção que levantamos – em especial o segundo o e último – merecem mais atenção, pois podem ser um pouco confusos (especialmente para iniciantes).

Afirmamos que, por conveniência, os números de elementos novos devem ser o mesmo número de elementos substituídos, pois caso contrário a alteração pode acontecer de forma indesejada.

ATENÇÃO: O que vamos explicar agora é um pouco confuso e envolve alguns conceitos que ainda não estudamos. **Não fique preocupado se não conseguir entender completamente nesse momento. O objetivo é apenas demonstrar que devemos ter cuidado quando formos realizar a substituição do slicing para que o programa não apresente resultados indesejados.**

Vamos usar o nosso exemplo das compras para demonstrar o que estamos querendo dizer. Desejamos alterar o intervalo [0:3] (os 3 primeiros elementos). O que aconteceria se pegássemos esse intervalo todo com 3 elementos e substituíssemos somente por um elemento?

Hora do teste! Agora esse intervalo [0:3] será substituído somente por “bolo”:

```python
compras = ["abacaxi", "peito de frango", "suco de laranja", "sabonete",
           "ovos", "papel higiênico", "leite", "abacate", "pasta de dente"]

compras[0:3] = "bolo"
print(compras)

```

Executando:

```
['b', 'o', 'l', 'o', 'sabonete', 'ovos', 'papel higiênico', 'leite', 'abacate', 'pasta de dente']

Process finished with exit code 0
```

Observe a bagunça que aconteceu: o Python pegou a string “bolo” e, como ela é iterável, considerou cada caractere dele como um elemento que irá substituir os elementos que fazem parte do intervalo do fatiamento.

Assim, “abacaxi” foi substituído por “b”, “peito de frango” foi substituído por “o”, “suco de laranja” foi substituído por “l” e o último “o” da string “bolo” foi adicionado na lista sem substituir nada, aumentando em 1 o tamanho da nossa lista!

Por que isso aconteceu? Porque passamos uma string para substituir um intervalo de 3 elementos. O Python esperava que fosse passado uma iterável com 3 elementos nela, de forma que ele pudesse pegar cada elemento e substituir 1 por 1: o primeiro do intervalo com o primeiro da atribuição, o segundo do intervalo com o segundo da atribuição e o terceiro do intervalo com o terceiro da atribuição.

Como passamos somente uma string (que é um iterável) com 4 caracteres ele entendeu o seguinte: “o programador deseja que eu itere sobre essa string e faça as substituições com base nela. Como eram 3 elementos a serem substituídos e a string tinha 4 caracteres, vou realizar as substituições e adicionar esse quarto elemento na lista, para não haver perda de informação.”

É por essa razão é que dissemos que é prudente passar o mesmo número de valores novos para aqueles que estamos querendo substituir.

Também afirmamos que os valores que passamos para a alteração por slicing devem ser iteráveis ou estarem contidos em um iterável. Vamos explorar melhor essa ideia, pois irá nos ajudar até mesmo a entender melhor o comportamento que acabamos de explicar acima.

Vamos fazer ciência! Vamos colocar essa string “bolo”, que é um iterável, dentro de outro iterável e passar como valor para substituir o intervalo. Mas onde podemos encontrar uma estrutura que seja iterável para colocar a nossa string nela?

Felizmente nós já conhecemos uma estrutura que atende a esses requisitos. Estamos estudando ela agora mesmo: as listas! Colocando em prática, então:

```
1   compras = ["abacaxi", "peito de frango", "suco de laranja", "sabonete",
2              "ovos", "papel higiênico", "leite", "abacate", "pasta de dente"]
3
4   compras[0:3] = ["bolo"]
5   print(compras)
6
```

Console:

```
['bolo', 'sabonete', 'ovos', 'papel higiênico', 'leite', 'abacate', 'pasta de dente']

Process finished with exit code 0
```

Bem melhor agora! O que aconteceu? Os três primeiros elementos foram substituídos por um só (“bolo”). Agora a lista está menor, pois 2 deles (“peito de frango” e “suco de laranja”) foram excluídos, pois não havia nada a ser colocado no lugar deles.

Por que isso aconteceu? Dessa vez passamos uma lista contendo a string “bolo” como seu único elemento para fazer a substituição. O Python entendeu que recebeu um iterável (uma lista) e entendeu que deveria substituir 3 elementos da lista compras pelos elementos que estavam dentro de [“bolo”] (ou seja, somente “bolo”). Assim, ele não iterou sobre a string “bolo”, mas sim sobre a lista [“bolo”].

Como somente havia um elemento dentro da lista para substituir 3, o Python pensou o seguinte: “o programador quer que eu substitua os 3 primeiros elementos da lista compras pelos elementos que estão na lista [“bolo”]. Como só tem um elemento para substituir 3, vou colocar “bolo” no lugar de “abacaxi” e deletar “peito de frango” e “suco de laranja”, pois não tem nada que possa substituí-los.

Agora vem um conceito novo. Vamos lembrar da substituição que fizemos da melhor forma: 3 valores substituindo 3 elementos da lista:

```
1    compras = ["abacaxi", "peito de frango", "suco de laranja", "sabonete",
2               "ovos", "papel higiênico", "leite", "abacate", "pasta de dente"]
3
4    compras[0:3] = "refrigerante", "pizza", "sorvete"
```

Nesse caso funciona perfeitamente, pois temos 3 itens substituindo 3 elementos. Mas como o Python sabe que é pra pegar cada string em vez de iterar sobre elas e pegar todos os caracteres de todas? Afinal, não mandamos as strings “refrigenrante”, “pizza” e “sorvete” dentro de uma lista [].

E aqui que vai um segredo: realmente não mandamos dentro de uma lista, mas, quando o Python vê esses 3 itens que vão substituir os elementos de “compras” ele enxerga uma tupla, que é outra estrutura de dados que estudaremos mais a frente.

E a tupla é iterável. Então cada elemento dessa tupla será usado para substituir o elemento correspondente da lista compras. **Ainda não estudamos tuplas e vamos ver os conceitos mais para frente. Não se preocupe se ainda está confuso. O objetivo agora é apenas entender o funcionamento do slicing, percebendo que devemos ter**

cuidado para não sermos surpreendidos por comportamento inesperado do programa.

Futuramente, quando tivermos estudado mais conceitos e o funcionamento de outras estruturas de dados, poderemos entender melhor o significado do que foi exposto agora.

2.4 Adicionando Elementos

Uma das possibilidades que a mutabilidade proporciona é a capacidade de adicionarmos novos elementos na lista. Assim, ela aumentará de tamanho para acomodar os novos itens.

O Python nos oferece métodos específicos para adição de novos elementos. Estudaremos os principais: append(), insert() e extend().

2.4.1 Método append()

O método append() permite adicionar elementos ao final da lista. É o método mais simples e o mais utilizado no dia a dia do programador. O seu funcionamento é bem direto: ele adiciona apenas um elemento por vez ao final da lista. Vejamos na prática:

```
1    numeros = [1, 2, 3]
2    numeros.append(4)
3
4    print(numeros)
5
```

Aqui temos a lista “numeros” com os elementos 1, 2 e 3. Usamos o append para adicionar o 4 na lista. Ao executar o programa o comando print() irá exibir a lista com os elementos 1, 2, 3 e 4:

```
[1, 2, 3, 4]

Process finished with exit code 0
```

Não existe dificuldade. É só isso mesmo. Usamos a sintaxe nomedalista.append(o que desejamos inserir na lista).

ATENÇÃO: O método append() somente permite a inclusão de 1 elemento por vez. Caso o programador tente passar mais de um elemento o programa exibirá erro:

```
1   numeros = [1, 2, 3]
2   numeros.append( __object: 4, 5)
3
4   print(numeros)
5
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    numeros.append(4, 5)
    ~~~~~~~~~~~~~~~^^^^^^
TypeError: list.append() takes exactly one argument (2 given)

Process finished with exit code 1
```

Erro de tipo: o método append() recebe exatamente um argumento (foram dados 2).

2.4.2 Método insert()

O método insert() oferece um pouco mais de controle. Ele permite ao programador adicionar um elemento na lista em um índice específico, ou seja, o programador escolhe a posição na lista onde o novo item será adicionado.

Esse método tem a seguinte sintaxe: nomedalista.insert(índice,valor). Passamos para o método primeiro o índice que o elemento a ser adicionado vai ocupar (então tem de ser um número inteiro) e depois da vírgula o que queremos adicionar mesmo.

Exemplo:

```
numeros = [1, 2, 3]
numeros.insert( _index: 1,  _object: 4)

print(numeros)

```

```
[1, 4, 2, 3]

Process finished with exit code 0
```

Observe que duas coisas aconteceram:

a) foi adicionado o elemento 4 na posição 1, da forma que mandamos ser feito;
b) O 4 entrou no lugar que era do 2. Então todos os outros números avançaram uma posição na lista para acomodar o 4.

Toda vez que usarmos insert() para adicionar algo no começo ou no meio da lista isso vai acontecer: o novo elemento vai ocupar a posição determinada e todos os elementos seguintes serão deslocados para a direita (avançar uma posição).

Assim como o append(), o insert() somente permite inserir um valor novo por vez!

ATENÇÃO: Se tentarmos usar um índice que não existe para o insert(), o elemento novo será adicionado no final da lista, se esse índice for positivo ou no início da lista, se esse índice for negativo. Exemplos:

Fim da lista:

```
numeros = [1, 2, 3]
numeros.insert( _index: 15,  _object: 4)

print(numeros)

```

```
[1, 2, 3, 4]

Process finished with exit code 0
```

Como não existe posição 15 para essa lista (ela vai até 2), o insert(15,4) adicionou o 4 na posição 3, ou seja, no final da lista (funcionou de forma semelhante ao append()).

Início da lista:

```
1    numeros = [1, 2, 3]
2    numeros.insert(-15,  _object: 4)
3
4    print(numeros)
5
```

```
[4, 1, 2, 3]

Process finished with exit code 0
```

Como não existe posição -15 nessa lista (de trás para frente ela vai até -3), o comando insert(-15,4) adicionou o 4 na posição -4, ou seja, no início da lista.

2.4.3 Método extend()

O método extend() é usado para adicionar vários elementos ao final da lista de uma só vez. Para funcionar, é necessário que mandemos um iterável, pois o método vai percorrer os elementos desse iterável e adicioná-los um a um no fim da lista e na mesma ordem que foram passados.

A sintaxe é: nomedalista.extend(iterável).

Com o exemplo fica bem fácil de entender:

```
numeros = [1, 2, 3]
numeros.extend([4, 5, 6])

print(numeros)

```

```
[1, 2, 3, 4, 5, 6]

Process finished with exit code 0
```

Como parâmetro do extend() passamos a lista contendo 4, 5 e 6. O extend() iterou sobre essa lista e adicionou à lista “numeros” todos esses elementos.

Outro exemplo de funcionamento:

```
numeros = [1, 2, 3]
numeros.extend([4, 5, 6])

mais_numeros = [7, 8, 9]
numeros.extend(mais_numeros)

print(numeros)

```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]

Process finished with exit code 0
```

Criamos outra lista chamada “mais_numeros” contendo 7,8 e 9. Passamos “mais_numeros” como parâmetro do extend() – extend(mais_numeros). O método iterou sobre os valores de “mais_numeros” e adicionou os seus elementos, na ordem em que apareceram, na lista “numeros”.

Por fim um exemplo envolvendo strings:

```
numeros = [1, 2, 3]
numeros.extend("quatro")

print(numeros)

```

```
[1, 2, 3, 'q', 'u', 'a', 't', 'r', 'o']

Process finished with exit code 0
```

Como as strings são iteráveis, o extend() pegou cada caractere dela e adicionou na lista “numeros”. Para passar a string “quatro” inteira como elemento da lista “numeros” basta colocar a string dentro de outro iterável (uma lista por exemplo):

```
numeros = [1, 2, 3]
numeros.extend(["quatro"])

print(numeros)

```

```
[1, 2, 3, 'quatro']

Process finished with exit code 0
```

Agora “quatro” foi inteiro pra lista porque o extend() iterou sobre a lista que continha a string (e não sobre a string em si). Como “quatro” era o único elemento daquela lista, ele foi adicionado na lista “numeros” em sua totalidade e não caractere por caractere.

ATENÇÃO: Os métodos append() e extend() são bem diferentes. O append() passa um único valor ao fim da lista (adiciona o elemento do jeito que ele está). Já o extend() percorre um iterável e vai adicionando o seu conteúdo ao fim da lista. Veja o que acontece quando comparamos os métodos adicionando a lista [4, 5, 6] ao final de “numeros”:

Com o extend():

```
numeros = [1, 2, 3]
numeros.extend([4, 5, 6])

print(numeros)

```

```
[1, 2, 3, 4, 5, 6]

Process finished with exit code 0
```

Com o append():

```
numeros = [1, 2, 3]
numeros.append([4,5,6])

print(numeros)

```

```
[1, 2, 3, [4, 5, 6]]

Process finished with exit code 0
```

Enquanto o extend() adicionou os números 4, 5 e 6 ao fim da lista, o append() adicionou a lista [4,5,6] – que é uma coisa só – ao final da lista. Nesse caso ficamos com 1, 2, 3 e uma outra lista (que contém 4, 5 e 6).

2.5 Removendo Elementos

A mutabilidade das listas também comporta a remoção de elementos. Podemos remover itens da lista, diminuindo o seu tamanho.

Os principais métodos do Python para a remoção de elementos de uma lista são: remove(), pop(), del, clear().

2.5.1 Método remove()

Esse método permite que removamos um valor da lista.

A sua sintaxe é: nomedalista.remove(valor).

O programador informa o valor que deseja remover da lista. Esse método percorre a lista até encontrar a primeira ocorrência desse valor e o exclui. Exemplo:

```
1    frutas = ["banana", "manga", "laranja"]
2    frutas.remove("laranja")
3
4    print(frutas)
5
```

Console:

```
['banana', 'manga']

Process finished with exit code 0
```

O elemento “laranja” foi removido. A lista agora somente possui dois elementos (de tamanho 3 passou para tamanho 2).

Conforme mencionamos, o método remove apenas a primeira ocorrência, ou seja, no nosso caso de remove(“laranja”), a primeira string “laranja” que o Python encontrar será removida e o método encerra a sua execução. O remove() não ira remover todas as ocorrências que existem na lista. Vejamos:

```
1    frutas = ["laranja", "banana", "manga", "laranja", "abacate", "laranja"]
2    frutas.remove("laranja")
3
4    print(frutas)
5
```

Executando:

```
['banana', 'manga', 'laranja', 'abacate', 'laranja']

Process finished with exit code 0
```

Somente a “laranja” que estava na posição 0 (foi a primeira a ser encontrada) é que foi removida. Ainda temos “laranja” nas posições 2 (era 3 antes da remoção) e 4 (era 5 antes da remoção).

Com isso, também percebemos que quando removemos um item, os elementos seguintes são deslocados para a esquerda (perdem uma posição). Essa reorganização é automática.

E o que acontece se o remove() não encontrar na lista o valor que pedimos para remover? Vamos testar!

```
1   frutas = ["laranja", "banana", "manga"]
2   frutas.remove("abacate")
3
4   print(frutas)
5
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    frutas.remove("abacate")
    ~~~~~~~~~~~~~~^^^^^^^^^^
ValueError: list.remove(x): x not in list

Process finished with exit code 1
```

O programa parou de funcionar por erro! Tentamos remover “abacate”, mas o método remove() não encontrou a string “abacate” na lista. Então recebemos um erro de valor explicando que “abacate” não está na lista e por isso o comando remove() não pôde ser executado.

2.5.2 Método pop()

O método pop() permite remover um elemento usando a posição que ele ocupa na lista.

Sintaxe: nomedalista.pop(índice).

Enquanto o remove() é usado para remover a primeira ocorrência do valor que passamos para ele, o pop() remove o valor que está no índice que o programador indicar. É bem simples:

```
fracoes = [1.2, 1.5, 7.3, 1.5, 2.1]
fracoes.pop(3)

print(fracoes)

```

```
[1.2, 1.5, 7.3, 2.1]

Process finished with exit code 0
```

Demos a ordem para que o método pop() removesse da lista o elemento de índice 3. Como no índice 3 temos o número 1.5, ele é que foi removido. Com isso a lista ficou menor e o 2.1 passou da posição 4 para a posição 3, pois a lista se reorganizou automaticamente.

Observe também que o 1.5 que ocupa a posição 1 foi ignorado pelo método pop(), pois ele não estava buscando um valor para excluir (quem faz isso é o remove()), mas sim o índice 3, ou seja, a posição da lista.

O pop() tem um comportamento especial: é possível usar pop() sem passar nenhum índice para ele. Nesse caso, o método está programado para remover o último elemento da lista. Exemplo:

```
fracoes = [1.2, 1.5, 7.3, 1.5, 2.1]
fracoes.pop()

print(fracoes)

```

```
[1.2, 1.5, 7.3, 1.5]

Process finished with exit code 0
```

Não informamos o índice do elemento que queríamos excluir. Então, por padrão, o pop() remove o último elemento da lista. Esse comportamento do pop() é muito utilizado no dia a dia da programação.

Mas e se o programador tentar remover um índice que não existe na lista? O programa vai dar erro:

```
1    fracoes = [1.2, 1.5, 7.3, 1.5, 2.1]
2    fracoes.pop(7)
3
4    print(fracoes)
5
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    fracoes.pop(7)
    ~~~~~~~~~~~^^^
IndexError: pop index out of range

Process finished with exit code 1
```

Erro de índice! O índice que passamos para o método pop() está fora do intervalo da lista (nossa lista tem índices de 0 a 4 e tentamos excluir o que está no índice 7, que não existe).

Por fim, o pop() tem mais uma funcionalidade interessante: ele permite que armazenemos o que foi removido em uma variável. Com o exemplo fica fácil entender:

```
1    fracoes = [1.2, 1.5, 7.3, 1.5, 2.1]
2    elemento_removido = fracoes.pop()
3
4    print(elemento_removido)
5
```

Console:

```
2.1

Process finished with exit code 0
```

Vamos entender melhor o que foi feito. Dessa vez criamos uma variável chamada “elemento_removido” e atribuímos a ela o valor de fracoes.pop(). O que pedimos ao Python foi o seguinte: pegue a lista fracoes e execute fracoes.pop() para remover o último elemento dessa lista. Em seguida, armazene o valor que foi removido na variável “elemento_removido”.

Esse comportamento de remover o elemento e retornar o que foi removido é algo muito utilizado pelos programadores.

2.5.3 Comando del

O comando del pode ser usado para deletar elementos de uma lista. Ele não é propriamente um método de listas, pois funciona em outras circunstâncias também. É um comando da linguagem Python que permite excluir elementos utilizando índices da lista, intervalos (slicing) ou apagar a lista inteira da memória do programa (deletar).

Sintaxe do comando del na lista:

a) Excluir um elemento: del nomedalista[índice];
b) Excluir intervalo: del nomedalista[início:fim];
c) Excluir a lista inteira: del nomedalista.

Vamos estudar todos os casos! Começando pela exclusão de um elemento específico:

```
1    notas = [10.00, 7.25, 5.50, 7.50]
2    del notas[2]
3
4    print(notas)
5
```

```
[10.0, 7.25, 7.5]

Process finished with exit code 0
```

O comando del notas[2] deletou o valor que ocupava o índice de número 2 na lista “notas”. Com isso, removemos a nota 5,50 da nossa lista.

Excluindo um intervalo:

```
1    notas = [10.00, 7.25, 5.50, 7.50, 3.75, 0, 8.00]
2    del notas[1:6]
3
4    print(notas)
5
```

```
[10.0, 8.0]

Process finished with exit code 0
```

Usando slicing, removemos todas as notas que estavam no intervalo do índice 1 (incluído) ao índice 6 (não incluído) da lista. Com isso, removemos os valores que ocupavam os índices 1, 2, 3, 4 e 5.

Excluindo a lista:

```
1    notas = [10.00, 7.25, 5.50, 7.50, 3.75, 0, 8.00]
2    del notas
3
4    print(notas)
5
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 4, in <module>
    print(notas)
          ^^^^^
NameError: name 'notas' is not defined

Process finished with exit code 1
```

Obtemos uma mensagem de erro ao tentar printar a lista “notas”. Erro de nome. O nome “notas” não está definido. Isso aconteceu porque o comando del não esvaziou a lista, mas sim a deletou completamente da memória. A lista não existe mais e por isso não há como acessá-la ou manipulá-la.

ATENÇÃO: Sempre que possível o programador deve priorizar o uso dos métodos específicos para cada operação. Quando o objetivo é remover um valor da lista, prefira o método remove(). Quando for necessário remover um elemento com base em seu índice, use o pop(). Dê preferência ao comando del quando o objetivo for deletar a lista da memória do programa ou quando for necessário remover elementos usando slicing.

2.5.4 Método clear()

O método clear() é utilizado para esvaziar a lista, ou seja, remover todos os seus elementos. **Diferente do comando del, o clear() não apaga a lista, mas apenas a esvazia.** Assim, essa lista ainda existe na memória do computador e pode ser acessada e modificada pelo programa.

Sintaxe: nomedalista.clear().

Vamos testar:

```
1    notas = [10.00, 7.25, 5.50, 7.50, 3.75, 0, 8.00]
2    notas.clear()
3
4    print(notas)
5
```

```
[]

Process finished with exit code 0
```

Dessa vez não recebemos mensagem de erro! Conseguimos acessar a lista "notas" usando o comando print. O clear() esvaziou a lista e o console está exibindo uma lista vazia. O programador ainda pode acessar e manipular essa lista.

A lista não foi deletada, mas apenas esvaziada. Ela ainda existe na memória durante a execução do programa, mas não possui nenhum elemento.

2.6 Métodos Adicionais

Além dos métodos já estudados, as listas contam com outros métodos muito úteis que merecem ser estudados. Vamos abordar os métodos sort(), reverse(), count() e index().

2.6.1 Método sort()

Esse método serve para colocar a lista em ordem, ou seja, ele nos permite modificar a lista para que os seus elementos sejam organizados em ordem crescente ou decrescente.

A sintaxe para ordenar os elementos em ordem crescente é: nomedalista.sort().

Para ordenação em ordem decrescente: nomedalista.sort(reverse=True).

Vamos testar usando uma lista numérica:

```
1  numeros = [97, 1043, 854, 53, 2, 65, 12]
2  numeros.sort()
3  print(f"Lista em ordem crescente: {numeros}")
4
5  numeros.sort(reverse=True)
6  print(f"Lista em ordem decrescente: {numeros}")
7
```

```
Lista em ordem crescente: [2, 12, 53, 65, 97, 854, 1043]
Lista em ordem decrescente: [1043, 854, 97, 65, 53, 12, 2]

Process finished with exit code 0
```

O programa foi executado da forma esperada. O método sort() modificou a lista, reorganizando os seus elementos para posicionar os números em ordem crescente. Posteriormente, o método sort() foi chamado novamente, mas dessa vez passando o parâmetro reverse=True.

Com isso, a lista que até agora estava em ordem crescente foi modificada novamente: os seus elementos foram trocados de posição para ordem decrescente.

Esse método funciona com listas cujos elementos sejam strings. Nesse caso a ordem considerada é a ordem alfabética:

```
1   compras = ["Maçã", "Alface", "Beterraba", "Salmão", "Limão", "Arroz"]
2   compras.sort()
3   print(compras)
4
```

```
['Alface', 'Arroz', 'Beterraba', 'Limão', 'Maçã', 'Salmão']

Process finished with exit code 0
```

As nossas compras foram colocadas em ordem alfabética.

ATENÇÃO: O método sort() pode apresentar comportamento inesperado ou erros se utilizado em listas com vários tipos diferentes como elementos. Vamos dar um exemplo com uma lista contendo inteiros e strings:

```
misturado = [30, "Python", 1, "Abelha"]
misturado.sort()
print(misturado)

```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    misturado.sort()
    ~~~~~~~~~~~~~~^^
TypeError: '<' not supported between instances of 'str' and 'int'

Process finished with exit code 1
```

Como podemos ver, tivemos um erro. O Python não consegue comparar string com int para saber qual é o maior ou menor para colocar em ordem.

2.6.2 Método reverse()

Esse método simplesmente inverte a lista. Ela é colocada de trás para frente, com modificação da posição de seus elementos. Diferente do sort(), não há nenhum tipo de ordenação.

Sintaxe: nomedalista.reverse().

É muito simples:

```
numeros = [97, 1043, 854, 53, 2, 65, 12]
numeros.reverse()
print(numeros)

```

```
[12, 65, 2, 53, 854, 1043, 97]

Process finished with exit code 0
```

A lista foi modificada e agora o primeiro elemento virou o último, o segundo virou o penúltimo e assim sucessivamente. A lista foi invertida e agora está de trás para frente.

ATENÇÃO: Não confundir o método reverse() com o parâmetro "reverse=True" que podemos passar para o método sort() para ordenar a lista de forma decrescente. São coisas totalmente diferentes apesar do nome.

2.6.3 Método count()

Usamos count() para descobrir o número de ocorrências de um elemento na lista, ou seja, quantas vezes o valor aparece na lista.

Sintaxe: nomedalista.count(valor).

É extremamente simples. O método indica o número de vezes que o valor aparece na lista, sendo que 0 significa o valor procurado não está presente na lista.

```
1    numeros = [1, 5, 7, 1, 9, 11, 1, 13, 15, 1]
2    quantidade = numeros.count(1)
3    print(quantidade)
4
```

```
4

Process finished with exit code 0
```

O método count() contou quantos "1" estão na lista "numeros" e isso foi armazenado na variável "quantidade". Ao printar "quantidade", obtivemos o resultado 4, pois o número 1 aparece quatro vezes na lista "numeros".

2.6.4 Método index()

O index() serve para indicar em qual posição o elemento aparece na lista, **em sua primeira ocorrência.**

Sintaxe: nomedalista.index(valor).

Vamos usar uma lista que tenha o número 1 repetido por várias vezes e tentar achar qual a sua posição usando o método index():

```
1    numeros = [32, 67, 1, 5, 7, 1, 9, 11, 1, 13, 15, 1]
2    posicao = numeros.index(1)
3    print(posicao)
4
```

```
2

Process finished with exit code 0
```

Veja que na variável “posicao” foi armazenado o valor 2. Isso representa a posição da primeira ocorrência do número 1 na lista. Ao encontrar a primeira ocorrência, o método encerra o seu funcionamento, de forma que os demais números 1 repetidos não são identificados.

ATENÇÃO: Pesquisar a posição de um valor que não esteja na lista usando o método index() gera erro:

```
1    numeros = [32, 67, 1, 5, 7, 1, 9, 11, 1, 13, 15, 1]
2    posicao = numeros.index(99)
3    print(posicao)
4
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    posicao = numeros.index(99)
ValueError: 99 is not in list

Process finished with exit code 1
```

Erro de valor: o valor (no caso 99) não está na lista.

2.7 Copiando Listas

O Python permite que copiemos listas usando o método copy() ou usando slicing. Vamos estudar esses dois métodos e também abordar um erro muito comum ao se tentar copiar listas.

2.7.1 Como NÃO Copiar Listas

O erro mais comum que é cometido ao tentar criar a cópia de uma lista é simplesmente atribuir a lista a ser copiada a uma nova variável. Vejamos:

```
1    numeros = [1,2,3]
2    copia = numeros
3    print(numeros)
4    print(copia)
5
```

```
[1, 2, 3]
[1, 2, 3]

Process finished with exit code 0
```

Aparentemente está tudo certo. A lista “copia” parece ser uma cópia exata da lista “numeros”. Mas, observem o comportamento inesperado do nosso programa quando tentamos alterar uma das listas. No exemplo, vamos adicionar o número 4 na lista “copia”:

```
1    numeros = [1,2,3]
2    copia = numeros
3    copia.append(4)
4    print(numeros)
5    print(copia)
6
```

```
[1, 2, 3, 4]
[1, 2, 3, 4]

Process finished with exit code 0
```

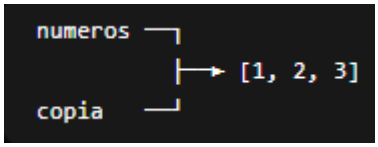
A nossa intenção era que o número 4 fosse adicionado na lista “copia”, mas, por algum motivo, quando fizemos essa adição ela também foi refletida na lista original “numeros”. Mas não era isso o que desejávamos fazer! O que está acontecendo?

O que aconteceu? Quando fazemos esse tipo operação, em que uma lista recebe outra por meio de atribuição, o que estamos fazendo, na verdade, é ligar as duas variáveis.

Para o Python elas não são duas listas independentes. O programa interpreta que “numeros” e “copia” são a mesma lista, apenas acessadas por variáveis diferentes. É como se a lista tivesse um nome e um apelido.

Por que isso acontece? Isso acontece porque tanto “numeros” quanto “copia” estão apontando para o mesmo endereço de memória. No computador, na memória principal (memória RAM), o programa está salvando a lista [1, 2 ,3] em um espaço determinado e indicando que esse espaço pode ser acessado tanto usando o nome “numeros” como “copia”

Visualmente seria algo desse tipo:



Com uma analogia, podemos pensar em um passageiro que entra num Uber e deseja ir ao shopping. Ele pode tanto falar o nome do shopping quanto informar o endereço da rua. O motorista vai chegar exatamente no mesmo lugar!

Para copiar listas de forma a criar verdadeiras cópias independentes, devemos usar o método copy() ou slicing. Essas formas criam listas independentes e que ocupam espaços diferentes na memória, de modo a evitar esse comportamento indesejado.

2.7.2 Método copy()

Esse método permite copiar uma lista e atribuir essa cópia a uma variável. A sua sintaxe é:

nomedalistanova = nomedalista.copy().

Assim, teremos duas listas iguais e independentes, pois uma cópia será criada. Vamos testar na IDE:

```
valores = [12.7, 32, 50.3, 40]
copia = valores.copy()
print(valores)
print(copia)

```

```
[12.7, 32, 50.3, 40]
[12.7, 32, 50.3, 40]

Process finished with exit code 0
```

Temos a lista “valores” e criamos uma nova lista chamada “copia”. Essa nova lista é uma cópia da lista “valores” e contém todos os seus elementos. Observem que o print de “valores” e de “copia” apresenta exatamente os mesmos elementos, mas são totalmente independentes. **Quando uma alteração é feita em uma delas, a outra não sofre essa alteração:**

```
valores = [12.7, 32, 50.3, 40]
copia = valores.copy()
copia.append(100)
print(valores)
print(copia)

```

```
[12.7, 32, 50.3, 40]
[12.7, 32, 50.3, 40, 100]

Process finished with exit code 0
```

O número 100 somente foi adicionado ao final da lista “copia” e a lista “valores” permaneceu inalterada

2.7.3 Cópia por Slicing

Além do método copy(), podemos simplesmente usar slicing para indicar qual parte da lista queremos copiar para a nossa nova lista (a nossa cópia). Exemplo:

copia = nomedalista[:]

Nesse caso, o slicing está indicando que queremos uma cópia completa da lista, ou seja, “copia” terá todos os elementos da lista “nomedalista”, pois o slicing indicou que queremos ela do início ao fim.

O resultado prático é o mesmo do copy() nesse caso:

```
1    valores = [12.7, 32, 50.3, 40]
2    copia = valores[:]
3    print(valores)
4    print(copia)
5
```

```
[12.7, 32, 50.3, 40]
[12.7, 32, 50.3, 40]

Process finished with exit code 0
```

Como podemos ver, obtivemos o mesmo resultado do exemplo no item anterior, quando usamos copy(). **As listas também são independentes:**

```
valores = [12.7, 32, 50.3, 40]
copia = valores[:]
copia.append(100)
print(valores)
print(copia)

```

```
[12.7, 32, 50.3, 40]
[12.7, 32, 50.3, 40, 100]

Process finished with exit code 0
```

Assim como no exemplo do copy(), o número 100 somente foi adicionado ao final da lista “copia” e a lista “valores” permaneceu inalterada.

Qual a diferença entre essas duas formas? O método copy() irá copiar a lista inteira. Já o slicing permite que copiemos apenas parte da lista.

2.8 Funções Úteis – len(), sum(), min(), max(), enumerate()

Além de todos os métodos específicos para listas, temos funções que funcionam para qualquer iterável, inclusive as listas. Essas funções são muito úteis e facilitam o dia a dia do programador. Vamos abordar as seguintes funções: len(), sum(), min(), max() e enumerate().

2.8.1 Função len()

Essa função já foi amplamente estudada na parte 2 desse tutorial. Ela simplesmente nos informa o tamanho do iterável, ou seja, quantos elementos ele tem. Vejamos um exemplo com listas:

```
numeros = [1, 2, 3, 4, 5]
tamanho = len(numeros)
print(tamanho)

```

```
5

Process finished with exit code 0
```

Como podemos ver, a função len() nos informou o tamanho da lista, que contém exatamente 5 números.

2.8.2 Função sum()

Essa função soma todos os números de uma lista numérica. Ela somente funciona se a lista é composta somente de inteiros e frações (int e float). Vejamos:

```
1    numeros = [1, 2.5, 3.14, 4, 5.27]
2    soma = sum(numeros)
3    print(soma)
4
```

```
15.91

Process finished with exit code 0
```

Todos os números da lista foram somados e armazenados na variável “soma”.

Teremos erro se tentarmos usar em listas com variáveis não numéricas:

```
1    numeros = [1, "dois", 3, "4", 5]
2    soma = sum(numeros)
3    print(soma)
4
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    soma = sum(numeros)
TypeError: unsupported operand type(s) for +: 'int' and 'str'

Process finished with exit code 1
```

Erro de tipo: é impossível somar um int com uma string.

2.8.3 Função min()

Essa função encontra o menor elemento de uma lista. Em listas numéricas, será o menor número. Já em listas de strings, será a primeira palavra em ordem alfabética.

Exemplo com lista numérica:

```
numeros = [1, 2.5, 3, 4, 5.6]
menor = min(numeros)
print(menor)

```

```
1

Process finished with exit code 0
```

Foi encontrado o menor valor dentro da lista: o número 1.

Exemplo com uma lista de strings:

```
numeros = ['um', 'dois e meio', 'três', 'quatro', 'cinco vírgula seis']
menor = min(numeros)
print(menor)

```

```
cinco vírgula seis

Process finished with exit code 0
```

Nesse caso o menor valor foi “cinco vírgula seis”, pois em ordem alfabética o “c” aparece antes das demais letras iniciais das outras strings. Ironicamente, o valor mínimo representa o maior número, já que 5.6 é o maior número da nossa lista.

ATENÇÃO: A função min() não funciona caso a lista seja composta por elementos que não possam ser comparados. Veja o que acontece quando tentamos usar min() numa lista que mistura números com strings:

```
numeros = [1, "dois", 3, "4", 5]
menor = min(numeros)
print(menor)

```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    menor = min(numeros)
TypeError: '<' not supported between instances of 'str' and 'int'

Process finished with exit code 1
```

2.8.4 Função max()

A função max() encontra o maior valor da lista. Para listas numéricas é o maior número. Já para listas de strings é a última palavra em ordem alfabética.

Exemplo com números:

```
numeros = [1, 2.5, 3, 4, 5.6]
maior = max(numeros)
print(maior)

```

```
5.6

Process finished with exit code 0
```

O número 5.6 realmente é o maior valor da nossa lista.

Exemplo com strings:

```
numeros = ['um', 'dois e meio', 'três', 'quatro', 'cinco vírgula seis']
maior = max(numeros)
print(maior)

```

```
um

Process finished with exit code 0
```

O valor “um” é o último em ordem alfabética da nossa lista, sendo, portanto, o valor máximo. Novamente a ironia: o menor valor numérico foi o maior valor alfabético.

ATENÇÃO: Assim como na função min(), a função max() também não funciona caso a lista seja composta por elementos que não possam ser comparados. Se tentarmos usar a função max() em uma lista que misture valores numéricos com strings o programa irá apresentar erro, assim como aconteceu no exemplo com min().

2.8.5 Função enumerate()

Essa função é usada para caso o programador precise da posição e do valor de cada elemento da lista de forma simultânea. A função percorre a lista e, a cada iteração, fornece duas informações: a posição do elemento e o valor armazenado nessa posição.

O enumerate(), por funcionar sob iteração, deve ser usado dentro de um laço de repetição, preferencialmente o laço for. A sintaxe é:

for posicao, valor in enumerate(lista):

    instruções

Com isso, a função enumerate() vai percorrer a lista e na primeira variável (“posicao” no nosso caso) o índice será salvo, enquanto na segunda variável (“valor” no nosso caso) a valor que está na posição será salvo.

Com um exemplo fica fácil entender. Vamos usar o enumerate() para percorrer uma lista e indicar a posição e o valor de cada elemento:

```
1   frutas = ['banana', 'manga', 'limão', 'maçã', 'abacate']
2   for pos, val in enumerate(frutas):
3       print(f"Posição: {pos}; Valor: {val}")
4
```

```
Posição: 0; Valor: banana
Posição: 1; Valor: manga
Posição: 2; Valor: limão
Posição: 3; Valor: maçã
Posição: 4; Valor: abacate

Process finished with exit code 0
```

Observem que a função altera o valor das variáveis conforme foi iterando pela lista. A posição (índice) sempre foi armazenada na primeira variável, que chamamos de "pos". Já os valores que ocupam as respectivas posições foram salvos na segunda variável "val".

A função enumerate() percorreu a lista até o fim, repetindo esse comportamento em cada iteração. Usamos um print() formatado para evidenciar o funcionamento da função.

ATENÇÃO: As variáveis para a posição e valor podem ter qualquer nome. Geralmente usamos "pos" e "val" porque evidenciam que desejamos salvar a posição e o valor, mas podemos usar qualquer nome. Também é muito comum usar "i" e "j", por exemplo. O programador está livre para nomear as suas variáveis conforme a conveniência.

2.9 Listas Aninhadas

As listas são estruturas de dados muito versáteis e potentes, que podem inclusive conter outras estruturas de dados como seus elementos, conforme mencionamos no item 2.1, **as listas podem até mesmo conter outras listas!**

É justamente essa situação de termos uma lista dentro de outra que chamamos de listas aninhadas. Exemplo de uma lista contendo outra lista:

```
1    compras = ["frutas", "legumes", "carne", "produtos de limpeza"]
2    gastos = [compras, "carro", "aluguel", "seguro"]
3
```

Nesse exemplo temos a lista “compras” composta por strings indicando o que compramos. Já a lista “gastos” é composta por strings representando os nossos gastos e, além dessas strings, a lista compras está dentro dela também.

Vamos ver o que acontece no console se executarmos print(gastos):

```
[['frutas', 'legumes', 'carne', 'produtos de limpeza'], 'carro', 'aluguel', 'seguro']

Process finished with exit code 0
```

Observem que toda a nossa lista de compras, inclusive com as chaves [] indicando que é uma lista, foi exibida no console e, em seguida, os demais elementos da lista “gastos”.

Como a lista “compras” está dentro da lista “gastos”, dizemos que temos uma lista aninhada.

Outro exemplo que temos é com números. É muito comum e faz parte do dia a dia da programação que encontremos listas numéricas dentro de outras listas numéricas. Veja o exemplo:

```
1    matriz = [[1, 2], [3,4]]
2
```

A lista “matriz” tem como seus elementos a lista [1, 2] e [3, 4]. Novamente estamos diante de listas dentro de listas.

Como identificar e entender listas aninhadas? A identificação é simples: sempre que tivermos uma lista dentro de outra lista estamos diante de uma lista aninhada.

Já para entender vamos pensar na seguinte analogia: imagine que determinada empresa tenha em sua sede um armário com três gavetas. Na primeira gaveta do

armário, foram armazenados materiais de escritório, como papel, lápis e caneta. A segunda gaveta contém fichas de clientes, histórico de ordens e estoque. Por fim, na terceira gaveta estão os documentos fiscais, livro caixa e comprovantes de pagamento.

Podemos representar essa situação usando uma lista aninhada em Python:

```python
gaveta_1 = ["papel", "lápis", "caneta"]
gaveta_2 = ["clientes", "histórico", "ordens"]
gaveta_3 = ["documentos", "livro caixa", "comprovantes"]
armario = [gaveta_1, gaveta_2, gaveta_3]

```

Ou, ainda, a mesma situação representada de uma forma mais usual:

```python
armario = [["papel", "lápis", "caneta"], ["clientes", "histórico", "ordens"],["documentos", "livro caixa", "comprovantes"]]

```

Cada lista dentro de “armario” representa uma das gavetas.

2.9.1 Acessando Elementos

Podemos acessar os elementos das listas aninhadas da mesma forma que fazemos nas listas comuns. Entretanto, quando acessamos um elemento, é possível que tenhamos outra lista ocupando aquela posição. Exemplo:

```python
exemplo = [[0, 1, 2], [3, 4, 5], [6, 7, 8]]
print(exemplo[1])

```

```
[3, 4, 5]

Process finished with exit code 0
```

O resultado do print() foi o elemento que está na posição 1 da lista “exemplo”. Isso representa a lista [3, 4, 5], que foi corretamente exibida no console.

Porém, o fato de ser outra lista que ocupa essa posição levanta a seguinte pergunta: mas e se o programador precisar acessar um elemento dessa lista interna? Como proceder?

A solução para essa necessidade é simples e intuitiva. Basta inserirmos a posição da lista interna logo após indicarmos a posição da lista externa. Essa é a sintaxe:

elemento = nomedalista[posicaoexterna][posicaointerna]

Com essa técnica conseguimos acessar diversos níveis (ou dimensões) da lista. Assim, vamos supor que desejamos printar no console o número 6. Para isso, temos de acessar a lista que contém esse número – seria o elemento na posição 2 da nossa lista "exemplo" (lista externa).

Posteriormente, precisamos indicar qual a posição do elemento que queremos acessar na lista [6, 7, 8] (lista interna). No caso, o número 6 está na posição 0 dessa lista.

Usando a sintaxe que estudamos:

```
1    exemplo = [[0, 1, 2], [3, 4, 5], [6, 7, 8]]
2    print(exemplo[2][0])
3
```

```
6

Process finished with exit code 0
```

Essa sintaxe funciona independente do número de listas internas. Veja um exemplo com três níveis:

```
array = [0, [1, [2, 3]]]
print(array[1][1][1])

```

```
3

Process finished with exit code 0
```

Posição 1 da lista “array” mais externa é a lista [1,[2, 3]] (uma lista que contém outra lista dentro dela). Já na posição 1 da lista intermediária temos a lista [2, 3] e, por fim, na posição 1 da lista mais interna temos o número 3.

Esse exemplo serve para notarmos algo importante: **uma lista com diversos níveis de aninhamento pode se tornar muito complicada de manipular e utilizar. Somente use estruturas complexas quando o programa precisar disso.**

2.9.2 Modificando Elementos

Todas as formas que estudamos sobre modificação de elementos da lista - adição, mudança, ou remoção - também podem ser usados nas listas aninhadas. A diferença é que temos de tomar cuidado com os diversos níveis.

A sintaxe para acessar os elementos irá nos ajudar a manipular as listas internas, mas todos os mesmos princípios se aplicam.

Vamos aos exemplos. Para isso faremos uso de uma lista que represente turma de determinado ano escolar. Dentro da turma temos outra lista que representa os alunos e as suas respectivas médias:

```
turma = [["Aline", 9.5], ["João", 10.0], ["Bruno", 8.0]]

```

Primeiro exemplo: o professor errou o nome de um aluno. Na verdade, não existe o aluno “João”, pois o seu nome é Carlos. Para modificar esse elemento basta usar a sintaxe que aprendemos:

```
turma = [["Aline", 9.5], ["João", 10.0], ["Bruno", 8.0]]
turma[1][0] = "Carlos"
print(turma)

```

```
[['Aline', 9.5], ['Carlos', 10.0], ['Bruno', 8.0]]

Process finished with exit code 0
```

“João”, que antes ocupava a posição [1][0], agora passa a ser “Carlos”.

Segundo exemplo: professor precisa adicionar um novo aluno na lista. É a Maria, que teve nota sete.

```
turma = [["Aline", 9.5], ["João", 10.0], ["Bruno", 8.0]]
turma[1][0] = "Carlos"
turma.append(["Maria", 7.0])
print(turma)

```

```
[['Aline', 9.5], ['Carlos', 10.0], ['Bruno', 8.0], ['Maria', 7.0]]

Process finished with exit code 0
```

Conseguimos adicionar uma nova lista ao final da lista “turma” contendo as informações que o professor precisava.

Terceiro exemplo: Adicionar informação faltante em uma lista interna. Considere a seguinte lista:

```
turma = [["Aline", 9.5], ["João", 10.0], ["Bruno"]]

```

O aluno “Bruno” está sem nota. Precisamos adicionar a média dele, que foi oito, na sua correspondente lista interna.

```
1    turma = [["Aline", 9.5], ["João", 10.0], ["Bruno"]]
2    turma[2].append(8.0)
3    print(turma)
4
```

```
[['Aline', 9.5], ['João', 10.0], ['Bruno', 8.0]]

Process finished with exit code 0
```

Executamos o método append() sobre elemento que ocupa a posição 2 da lista “turma”. Como temos uma lista nessa posição, o método funcionou corretamente e a nota oito foi adicionada ao final da lista interna.

ATENÇÃO: O programa teria dado erro se na posição 2 não houvesse outra lista, pois append() é um método específico das listas. Exemplo:

```
1    turma = [["Aline", 9.5], ["João", 10.0], "Bruno"]
2    turma[2].append(8.0)
3    print(turma)
4
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\lists.py", line 2, in <module>
    turma[2].append(8.0)
    ^^^^^^^^^^^^^^^^
AttributeError: 'str' object has no attribute 'append'

Process finished with exit code 1
```

Erro de atributo. Na posição 2 foi encontrada uma string (“Bruno”) e append() não é um método de string.

Recomendamos ao leitor que revise os itens anteriores sobre modificação de listas e os seus métodos e aplique esses ensinamentos nas listas aninhadas para praticar. Ficará evidente que os conceitos são os mesmos e o programador só precisa ter o

cuidado de indicar corretamente em qual lista (externa ou interna) é que a modificação pretendida deve ocorrer.

2.9.3 Representação de Matrizes

As listas aninhadas são utilizadas para representar matrizes. Vamos falar brevemente sobre o assunto, pois nosso foco é a programação, mas recomenda-se que o leitor estude e domine os conceitos matemáticos que estão por trás da programação.

Considere a seguinte matriz 2x2:

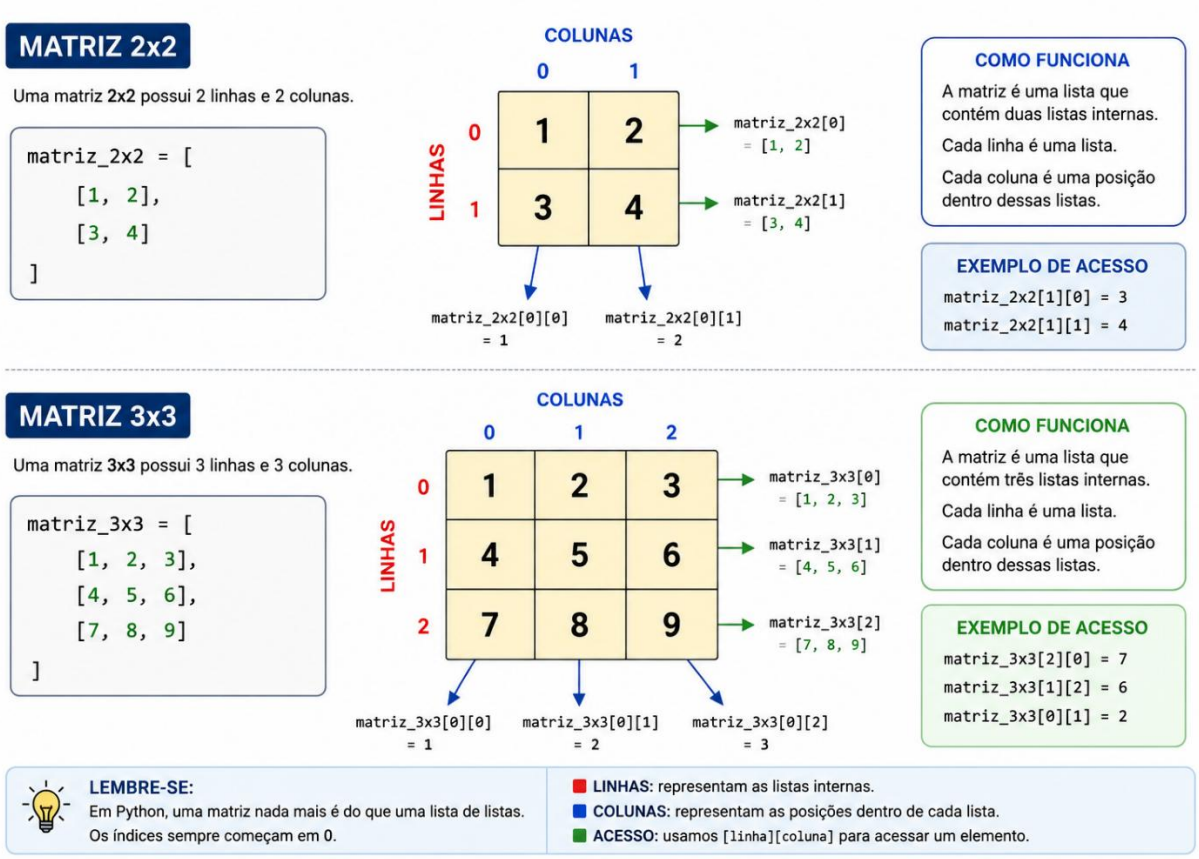
$$S = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Temos 2 linhas e 2 colunas. Nas linhas temos (1 e 2) e (3 e 4). Já nas colunas temos (1 e 3) e (2 e 4). Essa matriz pode ser corretamente representada no Python pela seguinte lista aninhada:

```
1    matriz = [[1, 2], [3, 4]]
```

Assim como na matriz S temos que 1 está na linha 1, coluna 1, na lista “matriz” 1 está na posição matriz[0][0]; 2 linha 1, coluna 2 e matriz[0][1]; 3 linha 2, coluna 1 e matriz [1][0]; 4 linha 2 coluna 2 e matriz[1][1].

A matriz 3x3 segue a mesma lógica. Observe a figura para facilitar a compreensão do conceito:



Isso significa que listas aninhadas podem ser úteis para representar matrizes ou dimensões. É possível adicionar novas dimensões realizando mais aninhamentos. Isso é extremamente útil quando estamos trabalhando com vetores, pois matematicamente são representados por matrizes.

ATENÇÃO: É possível representar matrizes com n dimensões, como por exemplo, 2x2x2 ou 4x4x4x4 ou qualquer outra que possamos imaginar. Entretanto, tome cuidado para não se perder na complexidade, pois quanto mais dimensões a sua matriz tiver, mais difícil será de representá-la corretamente e entender o código. Como sempre, a complexidade somente pode ser admitida quando realmente necessária!

2.9.4 Iteração de Listas Aninhadas

As listas, como estruturas de dados, são iteráveis e podem ser percorridas por um laço for para iterar sobre todos os seus elementos. Essa propriedade também se aplica às listas aninhadas.

Podemos usar um laço for básico para expor os elementos da lista aninhada, ou seja, exibir as listas que estão dentro da lista principal. Exemplo:

```
1   aninhada = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
2   for i in aninhada:
3       print(i)
4
```

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

Process finished with exit code 0
```

Já estamos muito acostumados com esse laço for. Já realizamos diversas operações desse tipo. No entanto, como podemos ver, o que foi printado no console foram as listas que compõem a lista principal “aninhada”. Não visitamos cada elemento individualmente na iteração.

Para fazer isso, vamos usar um laço for aninhado. Então precisaremos de um for para percorrer as listas, conforme já fizemos no exemplo acima, e outro para percorrer os elementos individualmente. Lembrando que caso esteja com dúvidas, reveja a parte 2 do tutorial, pois lá explicamos a fundo as estruturas de repetição.

Vamos usar o nosso for aninhado e ver o resultado:

```
1    aninhada = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
2    for lista in aninhada:
3        for elemento in lista:
4            print(elemento)
5
```

```
1
2
3
4
5
6
7
8
9

Process finished with exit code 0
```

O primeiro for (linha 2) percorreu a lista externa, como já fizemos no primeiro exemplo. Agora o for aninhado (linha 3), percorreu as listas internas, visitando os seus elementos individualmente.

ATENÇÃO: Normalmente, a quantidade de laços for necessária para percorrer os elementos individuais acompanha a dimensão da estrutura (quantidade de aninhamentos). É possível termos de utilizar diversos for aninhados para conseguir acessar todos os elementos.

2.10 List Comprehension

List comprehension é uma forma de criar uma lista de forma automática e a partir de dados já existentes. Para isso usamos um laço for e podemos até incluir condições.

Essa técnica de criar listas é útil e geralmente utilizada nos seguintes casos:

a) Criar uma nova lista a partir de outra;

b) Aplicar uma transformação aos elementos;
c) Filtrar elementos usando condições;
d) Escrever código mais compacto e legível.

A sintaxe é a seguinte:

nomedalista = [valor for elemento in iterável]

Caso haja condições a serem seguidas:

nomedalista = [valor for elemento in iterável if condições]

Os exemplos irão ajudar a entender o que está sendo feito. Vamos comparar a criação normal de listas que já estudamos com o método de list comprehension.

Vamos supor que o programador precise criar uma lista com números de 0 a 1.000. Usando o que nós já aprendemos e estudamos, provavelmente nosso código ficaria assim:

```
1    numeros = []
2    for i in range(1001):
3        numeros.append(i)
4
```

Com isso nós criamos uma lista vazia e, usando um laço for e o método append(), adicionamos os números de 0 a 1.000 na nossa lista.

Usando list comprehension, chegaremos no mesmo resultado, mas a sintaxe muda um pouco e faremos tudo apenas em uma mesma linha de código:

```
1    numeros = [i for i in range(0,1001)]
2
```

O resultado é exatamente o mesmo. Teremos uma lista de números do 0 ao 1.000! Só que dessa vez usamos apenas uma linha de código, o que pode tornar o nosso código mais legível e fácil de entender.

Um exemplo muito comum consiste em criar uma lista apenas com os números pares que estejam em outra lista usando list comprehension. Veja:

```
1    numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2    pares = [i for i in numeros if i%2 == 0]
3    print(numeros)
4    print(pares)
5
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[2, 4, 6, 8, 10]

Process finished with exit code 0
```

Novamente, em apenas uma linha de código já iteramos sobre todos os elementos da lista “numeros” e aplicamos a condição de ser par (o resto da divisão por 2 é 0). Ao mesmo tempo, os números que atendem à condição estabelecida são imediatamente adicionados na lista “pares”.

ATENÇÃO: List comprehension é apenas uma forma de criar listas. Não é algo novo e nem diferente de tudo o que estudamos, pois usando laço for tradicional chegamos ao mesmo resultado. Não é obrigatório que o programador faça uso dessa técnica a não ser que faça sentido para tornar o código mais legível e de fácil manutenção. **Evite usar list comprehension se isso for tornar o seu programa mais difícil de entender!**

CAPÍTULO 3 – Tuplas

3.1 Definição e Sintaxe

Tuplas são estruturas de dados capazes de armazenar diversos valores em uma única variável. Assim como as listas, as tuplas também conseguem operar com variados tipos, sejam eles strings, números, ou até mesmo outras estruturas de dados.

As tuplas são iteráveis, ordenadas e imutáveis, sendo que a imutabilidade é a principal característica. Assim, uma vez que uma tupla é declarada, não é mais possível modificá-la: não conseguimos alterar os seus elementos, nem adicionar ou remover itens.

Essa é a grande diferença entre as tuplas e as listas. Conforme formos estudando e desenvolvendo esse capítulo, ficará claro que ambas as estruturas são muito parecidas e versáteis, mas a principal diferença está na imutabilidade.

A sintaxe para declarar uma tupla é:

nomedatupla = (item1, item2, item3)

Como podemos ver, os seus elementos são separados por vírgulas, sendo esse o elemento que efetivamente define uma tupla. Porém, para deixar o código mais legível e indicar claramente a intenção do programador, normalmente utilizamos parênteses ().

Assim como nas listas, é possível criarmos uma tupla vazia:

nomedatupla = ()

Porém, lembre-se que ela não poderá ser alterada. Não podemos adicionar elementos novos após a sua declaração.

Vamos resumir as principais formas de se criar uma tupla:

```
1    classica = (1, 2, 3)
2    sem_parenteses = 1, 2, 3
3
```

A forma clássica é a sintaxe que ensinamos usando parênteses. Essa é a forma recomendada e é a mais utilizada. Caso passemos os valores separados por vírgulas e sem parênteses, o Python também entende que queremos criar uma tupla.

Mas, ressalta-se novamente: use preferencialmente a sintaxe nomedatupla = (item1, item2, item3).

Além disso, é possível converter outras estruturas de dados para o tipo tupla. Para isso usamos a palavra reservada tuple(). Exemplo:

```
1    array_numeros = [1, 2, 3, 4, 5]
2    print(array_numeros)
3    numeros = tuple(array_numeros)
4    print(numeros)
5
```

```
[1, 2, 3, 4, 5]
(1, 2, 3, 4, 5)

Process finished with exit code 0
```

Observem que o primeiro print() exibiu a lista “array_numeros” no console, inclusive com os colchetes []. Já o segundo print() feito após o casting (conversão) exibiu a tupla “numeros”, inclusive com os parênteses ().

Assim como ocorre com list(), a função tuple() percorre os elementos do objeto que está sendo convertido. Por isso, quando convertemos um iterável, cada elemento dele passa a ocupar uma posição na nova tupla:

```
palavra = "Python"
palavra_convertida = tuple(palavra)
print(palavra_convertida)

texto = "Hello World"
texto_convertido = tuple(texto)
print(texto_convertido)

```

```
('P', 'y', 't', 'h', 'o', 'n')
('H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd')

Process finished with exit code 0
```

Para evitar esse comportamento, devemos realizar a conversão já sobre uma lista que contenha a palavra ou o texto que queremos colocar na tupla. É bem similar ao que acontecia com as listas. Veja:

```
palavra = ["Python"]
palavra_convertida = tuple(palavra)
print(palavra_convertida)

texto = ["Hello World"]
texto_convertido = tuple(texto)
print(texto_convertido)

```

```
('Python',)
('Hello World',)

Process finished with exit code 0
```

ATENÇÃO: Diferente do que ocorre com as listas, quando usamos essa forma de conversão a tupla termina com uma vírgula. Isso ocorre porque as nossas tuplas do exemplo somente têm um único elemento. Para evitar ambiguidade, o Python insere

essa vírgula ao final, pois assim temos certeza de que estamos diante de uma tupla e não apenas de um valor colocado entre parênteses.

3.2 Acessando Elementos

A forma de acessar elementos numa tupla é idêntica à das listas, seguindo exatamente a mesma sintaxe:

nomedatupla[índice].

Tudo o que estudamos sobre acesso aos elementos de uma lista é aplicável à tupla, inclusive índices negativos e slicing. Segue breve revisão, mas, na dúvida, reveja o capítulo 2.

```
numeros = (0,1,2,3,4,5,6,7,8,9)
print(numeros[3]) # quarto elemento
print(numeros[9]) # último elemento
print(numeros[-1])  # último elemento usando índice negativo
print(numeros[:]) # tupla completa
print(numeros[::2]) # elementos de 2 em 2
print(numeros[::-1]) # invertida

```

```
3
9
9
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
(0, 2, 4, 6, 8)
(9, 8, 7, 6, 5, 4, 3, 2, 1, 0)

Process finished with exit code 0
```

3.3 Imutabilidade

As tuplas são imutáveis. Isso significa que não podemos alterá-las após a sua declaração. Não temos métodos para adicionar nem remover elementos. Também não temos como alterar os seus valores.

Por essa razão é que usamos tuplas quando estamos lidando com valores que não devem ser alterados. Elas existem justamente para resolver esse problema na programação: armazenar vários dados diferentes em uma mesma variável, mas garantir que esses dados serão preservados e ninguém poderá alterá-los, pois essa alteração poderia comprometer o bom funcionamento do programa.

ATENÇÃO: Existe uma interação entre tuplas e listas que causa aparente possibilidade de modificação. Isso ocorre quando temos uma lista dentro de uma tupla (um aninhamento). Não é possível alterar a tupla, mas a lista dentro dela poderá ser alterada. Exemplo:

```
1    compras = ["arroz", "feijão", "carne"]
2    imutavel = (12, compras, "Python", True)
3    compras.append("mudei dentro da tupla!")
4    print(imutavel)
5
```

```
(12, ['arroz', 'feijão', 'carne', 'mudei dentro da tupla!'], 'Python', True)

Process finished with exit code 0
```

Como podemos ver, não tem como alterar a tupla. Não há como excluir a lista "compras" de dentro dela, por exemplo. Entretanto, como lista é mutável, conseguimos alterar a lista "compras" adicionando uma nova string nela. E isso afetou a tupla, pois, apesar de não haver tecnicamente alteração na tupla em si, o seu objeto de referência é que foi modificado.

Ressaltamos que isso é uma situação excepcional e que escrever código que misture estruturas mutáveis com imutáveis deve ser feito com cautela e somente quando necessário. **Afinal, uma das principais vantagens das tuplas é justamente a garantia de que seus dados não serão alterados. Ao armazenar estruturas mutáveis dentro delas, parte dessa garantia acaba sendo perdida.**

3.4 Métodos de Tuplas

As tuplas possuem métodos para acessar os seus elementos e expor os índices que eles ocupam. Por serem imutáveis, não teremos métodos para adicionar, remover ou alterar itens.

Assim, teremos apenas os métodos count() e index(). Esses métodos irão funcionar exatamente como os métodos count() e index() que estudamos no capítulo 2.

O método count() irá contar o número de ocorrências de determinado elemento na tupla. Já o método index() irá retornar a posição que o elemento pesquisado ocupa na tupla, considerando apenas a primeira ocorrência desse elemento.

Sintaxe:

nomedatupla.count(elemento)

nomedatupla.index(elemento)

Como podemos ver, é exatamente o mesmo funcionamento dos métodos count() e index() que estudamos na lista. Vamos mostrar um exemplo de seu funcionamento para relembrar:

```
1    numeros = (0, 1, 0, 2, 1, 0)
2    print(numeros.count(1))
3    print(numeros.index(0))
4
```

```
2
0

Process finished with exit code 0
```

O método count() nos retornou 2, pois o número 1 aparece duas vezes nessa tupla. Já o método index() nos retornou 0, pois o número 0 está na primeira posição da tupla e o método index() trabalha apenas com a primeira ocorrência do valor pesquisado.

3.5 Funções Úteis

Assim como estudamos nas listas, as funções len(), sum(), max(), min(), e enumerate() também podem ser usadas nas tuplas.

Como já estudamos a fundo essas funções, apenas iremos relembrar o seu uso com exemplos nesse momento:

```
numeros = (0, 1, 2, 3, 4)
print(len(numeros)) # tamanho da tupla
print(sum(numeros)) # soma de todos os elementos da tupla
print(max(numeros)) # maior valor da tupla
print(min(numeros)) # menor valor da tupla
```

```
5
10
4
0

Process finished with exit code 0
```

Para usar a função enumerate() vamos precisar do laço for. Relembrando:

```
numeros = (0, 1, 2, 3, 4)
for pos, val in enumerate(numeros):
    print(f"Posição: {pos} Valor: {val}")
```

```
Posição: 0 Valor: 0
Posição: 1 Valor: 1
Posição: 2 Valor: 2
Posição: 3 Valor: 3
Posição: 4 Valor: 4

Process finished with exit code 0
```

3.6 Tuplas Aninhadas

O Pyhton permite o aninhamento de tuplas. Podemos ter tuplas dentro de outras tuplas, assim como estudamos com as listas. Entretanto, as semelhanças terminam quando chegamos na mutabilidade: as listas podem ser modificadas, já **as tuplas são imutáveis!**

A lógica do aninhamento e o acesso aos seus elementos seguem exatamente os mesmos princípios que estudamos nas listas. Assim, pouparemos o leitor de repetição desnecessária, sendo apenas necessário entender que o aninhamento de tuplas também é possível, mas sempre devemos levar em consideração a imutabilidade dessa estrutura de dados.

3.7 Quando Utilizar Tuplas?

Conforme estudamos, as tuplas e as listas são extremamente parecidas e podem ser utilizadas para finalidades semelhantes. A diferença central está justamente na mutabilidade das listas e na imutabilidade das tuplas.

Como podemos bem ver, as tuplas são imutáveis e não possuem métodos que permitam que sejam alteradas, reordenadas ou modificadas de qualquer maneira. Isso garante estabilidade aos dados que estão armazenados na tupla – tanto em relação aos seus valores quanto às suas posições na estrutura de dados.

Assim, as tuplas devem ser utilizadas quando o programador quer garantir essa imutabilidade: alterar a posição dos elementos ou os seus valores pode ser prejudicial ao programa e é indesejado.

ATENÇÃO: Não esquecer que existe a exceção da lista contida em uma tupla. A tupla não pode ser modificada, o que significa que a lista que está nela não pode ser substituída por outro elemento. Entretanto, conforme estudamos, essa lista que está dentro da tupla pode ser alterada! Reveja o que foi explicado ao final do item 4.3.

CAPÍTULO 4 – Tratamento de Exceções

4.1 Exceções

Exceção é definida como um evento que ocorre durante a execução de um programa e indica que alguma situação excepcional ou inesperada aconteceu. Essa situação gera interrupção no funcionamento do programa e o Python nos explica o que aconteceu.

No decorrer deste tutorial, desde a Parte 1, vimos diversos exemplos de exceções e de erros que "quebraram" o nosso programa. Sabemos que quando isso acontece, recebemos uma mensagem que indica o que ocorreu, em qual linha do código foi encontrado o problema e qual tipo de erro foi detectado.

Vamos refrescar a memória, pois neste capítulo vamos aprofundar um pouco mais sobre o que já sabemos de exceções. Veja o seguinte exemplo:

```
1    numero = int("abc")
2
```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\exceptions.py", line 1, in <module>
    numero = int("abc")
ValueError: invalid literal for int() with base 10: 'abc'

Process finished with exit code 1
```

Nesse exemplo, tentamos converter a string "abc" para um int. Isso gerou um erro e ativou o Traceback do Python. O Traceback, por sua vez, nos explica melhor o que aconteceu. Vamos analisar as informações que ele nos passou.

Em primeiro lugar, ele indicou qual arquivo deu problema, mostrando o seu nome e em qual pasta do computador ele está armazenado. Em seguida, ele nos indicou

o local no código em que ele detectou o problema – linha 1, especificamente no código “numero = int(“abc”).

Por fim, ele nos dá o tipo e o nome do erro que aconteceu, com uma explicação mais detalhada. No caso foi um ValueError e a explicação é que passamos um literal inválido (“abc”) para int() com base 10.

Em outras palavras, o Python, através do Traceback, está dizendo:

“Este programa que eu tentei executar e que está nesse lugar do computador encontrou um problema. O problema está na linha 1 quando o programador tentou fazer “numero = int(“abc”).

Isso me gerou um erro de valor, pois eu não consigo transformar “abc” em um valor numérico de base 10.”.

Com isso, sabemos qual o nome da exceção encontrada, o que ela significa, e qual foi a linha de código que gerou esse evento que levou à interrupção da execução do programa.

O próprio Python oferece ferramentas para detectar os problemas e informar ao programador o que está acontecendo. Isso facilita muito a correção de problemas para o bom funcionamento do programa.

4.2 Tratamento de Exceções

No decorrer desse tutorial, novamente desde a Parte 1, aprendemos com exemplos que exceções não tratadas irão interromper a execução do programa. Com isso, estudamos algumas formas de tratamento básico de dados para evitar que exceções ou comportamentos inesperados ocorram.

Por exemplo, vamos pegar um programa que receba do usuário dois números e realize a divisão entre eles. Sabemos que é impossível realizar divisão por 0 e, com tratamento básico de dados e utilização de estruturas de repetição, podemos informar ao usuário para que não passe 0 como divisor:

```
a = float(input("Digite o primeiro número: "))

while True:
    b = float(input("Digite o segundo número: "))

    if b == 0:
        print("Não é possível realizar uma divisão por zero. Digite outro número.")
    else:
        break

resultado = a / b
print(f"Resultado: {resultado}")

```

Nesse exemplo, utilizamos uma estrutura de decisão para verificar se o segundo número é igual a 0. Caso seja, não realizamos a divisão e informamos ao usuário que a operação não pode ser realizada. Caso contrário, a divisão é executada normalmente.

Essa é uma forma simples de antecipar um possível problema e impedir que ele aconteça. O programa não chega a tentar realizar uma operação que sabemos ser inválida.

Entretanto, nem sempre será possível prever e verificar antecipadamente todas as situações que podem provocar uma exceção. Além disso, existem situações em que queremos que o programa saiba como reagir caso uma exceção realmente ocorra.

É justamente para essas situações que o Python oferece mecanismos próprios para o tratamento de exceções.

A partir de agora, veremos como podemos fazer com que o programa identifique uma exceção durante sua execução e determine o que deverá ser feito quando ela ocorrer.

4.3 Estrutura try – except

O Python possui uma estrutura específica para o tratamento de exceções. Essa estrutura nos permite capturar as exceções e programar o que deve ser feito caso a exceção ocorra.

A sintaxe dessa estrutura é:

```
try:
        código que pode gerar uma exceção
except:
        código executado caso uma exceção ocorra
```

O funcionamento é bastante simples. O Python tentará executar o código colocado dentro do bloco try. Se nenhuma exceção ocorrer, a execução continuará normalmente e o bloco except será ignorado.

Por outro lado, se uma exceção ocorrer durante a execução do bloco try, o Python interromperá a execução desse bloco e passará para o bloco except, executando o código que estiver dentro dele.

Vamos mostrar um exemplo de funcionamento da forma mais simples possível dessa estrutura:

```python
try:
    numero = int(input("Digite um número inteiro: "))
    print(f"Você digitou o número {numero}.")
except:
    print("O valor informado não é um número inteiro válido.")

```

Dentro do bloco try temos um input() pedindo um número inteiro ao usuário. Esse input será convertido para int e armazenado na variável "numero". Se tudo correr bem, o número que o usuário digitou será printado.

Já o bloco except indica o que deve ser feito caso uma exceção aconteça. Se por algum motivo o usuário não passar um número inteiro válido, a execução do bloco try será interrompida e o bloco except será executado. No caso, será exibida uma mensagem informando que não foi digitado um número válido.

Vamos ver o que acontece se o usuário passar 5 como número:

```
Digite um número inteiro: 5
Você digitou o número 5.

Process finished with exit code 0
```

Observe que o programa funcionou normalmente. O valor passado foi convertido para int com sucesso e não houve nenhuma exceção. **O código dentro do bloco try foi executado em sua totalidade e o bloco except foi ignorado.**

Agora vamos ver o que acontece se o usuário passar a letra "r" como número:

```
Digite um número inteiro: r
O valor informado não é um número inteiro válido.

Process finished with exit code 0
```

Não é possível converter "r" para int! Aconteceu uma exceção (ValueError no caso). **Como ocorreu uma exceção, o Python interrompeu a execução do bloco try e**

foi para o bloco except, que dessa vez foi executado em sua totalidade exibindo a mensagem programada.

ATENÇÃO: Caso não houvéssemos tratado a exceção usando a estrutura try-except, o programa simplesmente teria quebrado e receberíamos aquela mensagem vermelha de erro do Traceback, indicando o tipo do erro e onde ele aconteceu.

Mas por que essa estrutura é útil se já conseguimos tratar os dados e prever problemas? Quando realizamos o tratamento de dados de forma preventiva, estamos apenas lidando com os problemas e situações que já conseguimos prever.

Isso é extremamente útil, mas nunca devemos subestimar a capacidade que os problemas têm para quebrar nosso código e nos surpreender! É extremamente comum que ocorram situações que não conseguimos prever e, consequentemente, nosso código não estará preparado para lidar com elas.

A estrutura try-except, por outro lado, nos permite preparar o código para abranger muito mais situações e de forma mais genérica ou mais específica (conforme a necessidade). É possível impedir que o programa quebre ainda que aconteça uma situação que não esteja expressamente prevista pelo nosso tratamento dos dados.

Conforme avançarmos no estudo dessa estrutura, a sua utilidade ficará cada vez mais evidente. Ela não serve para substituir o tratamento de dados, que sempre será uma boa prática. A estrutura try-except é uma outra ferramenta que pode e deve ser usada em conjunto para garantir o bom funcionamento do programa.

No nosso primeiro exemplo, apenas apresentamos uma forma muito básica do uso do try-except. Não fizemos nenhum tratamento dos dados e quando a exceção

ocorre o programa apenas exibe uma mensagem explicando o que aconteceu e encerra o seu funcionamento.

Vamos dar um exemplo mais avançado agora: vamos combinar try-except com tratamento de dados e, caso ocorra a exceção, o código não irá parar de funcionar, mas pedirá novo input do usuário:

```
while True:
    try:
        numero = int(input("Digite um número inteiro: "))

        if numero < 0:
            print("O número deve ser positivo.")
        else:
            print(f"Você digitou o número {numero}.")
            break

    except:
        print("O valor informado não é um número inteiro válido.")

```

Novamente, temos um bloco try pedindo ao usuário um input de número inteiro. Só que, dessa vez, desejamos que o número seja positivo!

Para isso, realizamos um tratamento de dados que detecta se o número digitado for menor que 0 e, nesse caso, exibe a mensagem que o número deve ser positivo. Em seguida, o programa pedirá o input novamente.

Como a estrutura try-except inteira está dentro do while True, caso uma exceção ocorra o bloco except será executado, mas o programa não será encerrado. Haverá nova tentativa de obter o input do usuário.

Exemplo quando digitamos -3:

```
Digite um número inteiro: -3
O número deve ser positivo.
Digite um número inteiro:
```

Nossa mensagem do tratamento de dados foi exibida e o programa ainda está funcionando! Novamente ele nos pede um número inteiro.

Vamos tentar passar “banana” como número:

```
Digite um número inteiro: -3
O número deve ser positivo.
Digite um número inteiro: banana
O valor informado não é um número inteiro válido.
Digite um número inteiro:
```

Não há como converter “banana” para int. O bloco except foi executado e a mensagem foi exibida. O programa ainda está funcionando e pedindo um número inteiro.

Vamos tentar 10 agora:

```
Digite um número inteiro: -3
O número deve ser positivo.
Digite um número inteiro: banana
O valor informado não é um número inteiro válido.
Digite um número inteiro: 10
Você digitou o número 10.

Process finished with exit code 0
```

Finalmente passamos um valor que não gera uma exceção e que também atende ao nosso tratamento de dados. O bloco try foi executado em sua totalidade, o bloco except foi ignorado, a execução prosseguiu normalmente e o programa foi encerrado.

4.3.1 Tratando Exceções Específicas

Até agora, utilizamos a estrutura try-except de uma maneira bastante genérica. Sempre que qualquer exceção ocorresse dentro do bloco try, o bloco except seria executado, independente de qual exceção!

Embora essa forma de utilização possa ser útil em algumas situações, ela possui uma limitação importante: nem sempre queremos tratar todas as exceções da mesma maneira. Diferentes exceções podem ocorrer por diferentes motivos e, consequentemente, podem exigir tratamentos diferentes.

Por exemplo, no nosso último programa, utilizamos a função int() para converter o valor recebido pelo usuário. Sabemos que, caso o usuário informe um texto que não possa ser convertido para um número inteiro, o Python gerará uma exceção do tipo ValueError.

Podemos indicar ao except que desejamos tratar especificamente esse tipo de exceção (ou qualquer outro que se mostre necessário).

A sintaxe é:

```
try:
    código que pode gerar uma exceção
except TipoDaExceção:
    código executado caso essa exceção ocorra
```

Assim, podemos escrever:

```python
try:
    numero = int(input("Digite um número inteiro: "))
    print(f"Você digitou o número {numero}.")

except ValueError:
    print("O valor informado não é um número inteiro válido.")

```

Agora, o bloco except não está preparado para tratar qualquer exceção. Ele foi configurado especificamente para tratar uma exceção do tipo ValueError.

Se o usuário digitar 10, por exemplo, nenhuma exceção ocorrerá. O bloco try será executado normalmente e o except será ignorado.

Se o usuário digitar "banana", a função int() não conseguirá realizar a conversão e ocorrerá um ValueError. Como temos um except ValueError, essa exceção será capturada e o código dentro do except será executado.

Essa forma de tratamento é mais precisa do que simplesmente utilizar um except sem especificar o tipo de exceção. Fazendo uso dela, é possível tratar cada exceção de forma individual.

ATENÇÃO: O tipo da exceção deve ser indicado pelo seu nome. Quando uma exceção ocorre, o Python informa seu tipo no Traceback. Esse é o mesmo nome que podemos utilizar para indicar ao except qual exceção desejamos tratar.

Por exemplo, vamos digitar “banana” num programa que tenta converter o input para um int sem qualquer tratamento de exceção:

```python
numero = int(input("Digite um número inteiro: "))
print(f"Você digitou o número {numero}.")

```

```
Digite um número inteiro: banana
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\exceptions.py", line 1, in <module>
    numero = int(input("Digite um número inteiro: "))
             ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'banana'

Process finished with exit code 1
```

O nome ValueError indica o tipo da exceção que ocorreu. Assim, podemos utilizá-lo diretamente: except ValueError. O nome do erro sempre estará indicado no Traceback.

Quando tratamos a exceção diretamente, isso permite que o programa trate especificamente as situações em que um ValueError ocorre, em vez de tratar indiscriminadamente qualquer exceção, como fizemos no item anterior usando apenas except sem indicar nenhuma exceção específica.

Essa distinção se torna especialmente importante quando um mesmo bloco try pode produzir diferentes tipos de exceção. Nesses casos, podemos determinar tratamentos diferentes para cada uma delas.

4.3.2 Tratando Múltiplas Exceções

A estrutura try-except admite vários blocos excepts. Isso nos permite tratar múltiplas exceções de forma individual e específica para cada uma delas. Cada bloco except associado ao try pode conter suas instruções específicas.

A sintaxe fica assim:

try:

código

except Tipodaexcecao:

código

except Tipodaexcecao:

código

Podemos ter quantos blocos except quanto forem necessários. Vamos aprofundar esse conhecimento.

Vamos pensar no seguinte exemplo: nosso programa irá receber dois números inteiros do usuário e fará a divisão entre eles. Então sabemos que tem duas possíveis exceções a serem capturadas: ValueError, pois o que o usuário digitar precisa ser um número; ZeroDivisionError, pois o divisor não pode ser zero.

Cada exceção terá tratamento próprio utilizando o que acabamos de aprender. Vamos escrever o código:

```
try:
    a = int(input("Digite o primeiro número: "))
    b = int(input("Digite o segundo número: "))

    resultado = a / b

    print(f"Resultado: {resultado}")

except ValueError:
    print("Você deve digitar apenas números inteiros.")

except ZeroDivisionError:
    print("Não é possível dividir por zero.")

```

Agora temos dois tratamentos diferentes para as exceções ValueError e ZeroDivisionError. A primeira será tratada com a exibição da mensagem "Você deve digitar apenas números inteiros." Já a segunda será tratada pela mensagem "Não é possível dividir por zero."

Vamos passar “banana” para ver o que acontece:

```
Digite o primeiro número: banana
Você deve digitar apenas números inteiros.

Process finished with exit code 0
```

Logo no primeiro número passado ao Python o problema foi identificado. Tivemos um ValueError, pois “banana” não pode ser convertida para int. O bloco except ValueError foi executado, pois essa exceção específica foi capturada e expressamente tratada pelo nosso programa.

Observe que o bloco do except ZeroDivisionError foi ignorado, pois essa exceção não aconteceu.

Vamos tentar dividir por 0 agora:

```
Digite o primeiro número: 2
Digite o segundo número: 0
Não é possível dividir por zero.

Process finished with exit code 0
```

Agora tivemos a ocorrência e captura do ZeroDivisionError. Como foi essa exceção que aconteceu e o nosso código estava preparado para ela, o bloco except ZeroDivisionError foi executado. Observe que, na mesma lógica que explicamos logo acima, dessa vez o bloco do except ValueError foi ignorado, pois nenhum ValueError aconteceu.

ATENÇÃO: Quando uma exceção ocorre dentro do try, o Python procura um except compatível com o tipo da exceção. Depois de encontrar e executar esse except, ele não executa os demais except daquele mesmo conjunto.

4.3.3 Múltiplas Exceções No Mesmo except

Um mesmo bloco except pode ser usado para tratar mais de uma exceção específica e ao mesmo tempo. Para isso, basta indicar todas as exceções que queremos tratar dentro de uma tupla no except.

A sintaxe é:

```
try:
        código
except (Tipodaexcecao1, Tipodaexcecao2)
        código
```

Isso é útil para quando queremos que múltiplas exceções recebam um mesmo tratamento. Vamos pegar o nosso exemplo do programa que faz a divisão de dois números:

```python
try:
    a = int(input("Digite o primeiro número: "))
    b = int(input("Digite o segundo número: "))

    resultado = a / b

    print(f"Resultado: {resultado}")

except ValueError:
    print("Você deve digitar apenas números inteiros.")

except ZeroDivisionError:
    print("Não é possível dividir por zero.")

```

Aqui, cada tipo de erro está sendo tratado de forma individual e com um bloco except próprio.

Vamos supor, entretanto, que o programador deseje que o ValueError e o ZeroDivisionError tenham exatamente o mesmo tratamento. Ele irá utilizar a sintaxe que estudamos e passará ambos os tipos de erro para o except numa tupla:

```python
try:
    a = int(input("Digite o primeiro número: "))
    b = int(input("Digite o segundo número: "))

    resultado = a / b

    print(f"Resultado: {resultado}")

except (ValueError, ZeroDivisionError):
    print("Operação inválida.")

```

Agora ambos os erros serão capturados e tratados por um mesmo bloco except.

ATENÇÃO: Quando desejamos que várias exceções tenham o mesmo tratamento, podemos agrupá-las em um mesmo except, passando os seus tipos dentro de tuplas. Por

outro lado, quando desejamos tratamento individualizado, devemos criar tantos blocos except quanto necessários, cada um capturando e tratando a exceção específica.

4.4 Cláusula Else

Podemos associar um else à estrutura try-except, que pode ser utilizada na forma try-except-else. A ideia é a seguinte: o bloco try tentará ser executado. Caso ocorra uma exceção, o bloco except é executado. **O bloco else somente é executado caso NÃO ocorra nenhuma exceção.**

Sintaxe:

```
try:
    código que pode gerar uma exceção
except:
    código executado se ocorrer uma exceção
else:
    código executado se NÃO ocorrer uma exceção
```

ATENÇÃO: Adicionar uma cláusula else à estrutura try-except é OPICIONAL. Não é obrigatório que toda estrutura try-except tenha um else. Todos os nossos exemplos anteriores funcionaram muito bem sem um bloco else.

Assim, o else somente é executado quando nenhuma exceção ocorre. Vamos dar um exemplo prático:

```python
try:
    a = int(input("Digite um número inteiro: "))

except ValueError:
    print("Você deve digitar apenas números inteiros!")

else:
    print(f"Você digitou o número {a}")

```

Nesse exemplo, o nosso programa recebe o input do usuário e imediatamente tenta convertê-lo para int. Caso a conversão não seja possível, acontece a exceção ValueError e o bloco except é executado. **Nessa hipótese o else é ingorado!**

Por outro lado, caso o valor passado pelo usuário seja convertido para int com sucesso, não houve nenhuma exceção, de forma que o bloco else será executado e o bloco except é que será ignorado.

Por que usar else? O else é utilizado quando queremos executar determinado código somente se nenhuma exceção tiver ocorrido no bloco try.

Embora seja possível colocar esse código dentro do próprio try, **o uso do else torna o código mais organizado, específico e fácil de entender.** Dessa forma, podemos manter no bloco try apenas as operações que podem gerar uma exceção e deixar no bloco else o código que deverá ser executado caso essas operações sejam concluídas com sucesso.

ATENÇÃO: Não confundir o else da estrutura if-else com o else da estrutura try-except. Na estrutura de decisão, o else tem ideia de caso contrário – se o if não acontecer, faça isso. **Já na estrutura try-except, o else tem a ideia de "serei executado caso nenhuma exceção ocorra".**

4.5 Cláusula Finally

Adicionamos um bloco finally à estrutura try-except quando desejamos que esse código sempre seja executado, independente da exceção ter ocorrido ou não.

O finally é algo totalmente opcional. Assim como o else, a estrutura não precisa dele para funcionar. O seu objetivo é apenas o que mencionamos: criar um bloco de código que deve ser executado tanto na hipótese de a exceção ter ocorrido quanto na hipótese de não ter acontecido a exceção.

A sintaxe é:

```
try:
    código que pode gerar exceção
except:
    código caso ocorra a exceção
finally:
    código a ser executado independente de ter exceção ou não
```

ATENÇÃO: Também podemos ter bloco finally quando temos bloco else. Ambos os blocos são opcionais e podem ser usados conforme a conveniência.

Vejamos o funcionamento na prática. Para isso vamos usar o mesmo exemplo do else, mas adicionando um bloco finally:

```python
try:
    a = int(input("Digite um número inteiro: "))

except ValueError:
    print("Você deve digitar apenas números inteiros!")

else:
    print(f"Você digitou o número {a}")

finally:
    print("Programa finalizado com sucesso!")

```

Vamos digitar “banana” no input para acontecer a exceção:

```
Digite um número inteiro: banana
Você deve digitar apenas números inteiros!
Programa finalizado com sucesso!

Process finished with exit code 0
```

A exceção aconteceu e foi capturada. O bloco except foi executado. Como houve exceção, o bloco else foi ignorado. **Independentemente de ter havido a exceção ou não, o finally foi executado!**

Vamos digitar 10 agora para que o programa rode sem exceção:

```
Digite um número inteiro: 10
Você digitou o número 10
Programa finalizado com sucesso!

Process finished with exit code 0
```

Dessa vez não tivemos exceção. O bloco except foi ignorado. Como não houve exceção, o bloco else foi executado. **Independentemente de ter havido a exceção ou não, o finally foi executado!**

Por que utilizar finally? O finally é particularmente útil quando existe alguma tarefa que precisa ser realizada independentemente de o código ter sido executado com sucesso ou ter ocorrido uma exceção.

ATENÇÃO: O bloco finally SEMPRE será executado, mesmo quando ocorrer uma exceção que não tenha sido tratada por um bloco except. Nesse caso, o Python executará primeiro o código presente no finally e, em seguida, a exceção continuará sua propagação. Como ela não foi tratada, o programa será interrompido e o Python exibirá a mensagem de erro e o respectivo Traceback:

```
try:
    a = int(input("Digite o primeiro número: "))
    b = int(input("Digite o segundo número: "))
    resultado = a/b
    print(resultado)

except ValueError:
    print("Valor digitado inválido.")

finally:
    print("Programa finalizado.")

```

```
Digite o primeiro número: 2
Digite o segundo número: 0
Programa finalizado.
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\exceptions.py", line 4, in <module>
    resultado = a/b
                ~^~
ZeroDivisionError: division by zero
```

Tentamos efetuar uma divisão por 0 e o nosso programa não tratou essa exceção. Por isso recebemos a mensagem de erro. **Porém, preste atenção que a mensagem "Programa finalizado." foi impressa no console, conforme programado no bloco finally!**

ATENÇÃO: A cláusula finally NÃO trata a exceção! o finally não impede a ocorrência ou a propagação de uma exceção; ele apenas garante que seu bloco seja executado antes que a execução continue ou a exceção seja propagada.

4.6 Acessando a Exceção com as

O Python permite associar a exceção capturada a uma variável. Para isso devemos usar a palavra reservada as. A sintaxe vai no bloco except:

try:

    código que pode gerar uma exceção

except tipodaexcecao as nomedavariavel

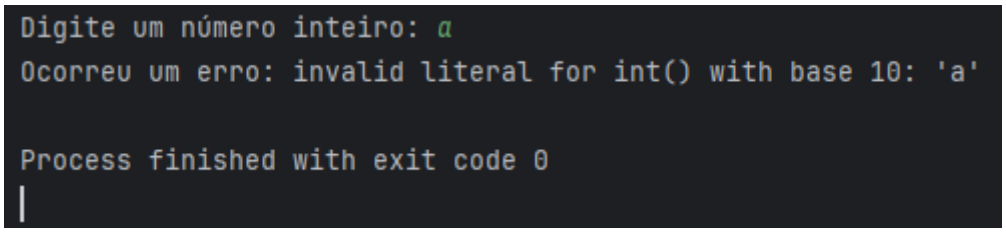
    código tratando a exceção

Com um exemplo fica mais fácil de entender:

```
1  try:
2      a = int(input("Digite um número inteiro: "))
3      print(a)
4
5  except ValueError as erro:
6      print(f"Ocorreu um erro: {erro}")
7
```

Atenção para a linha 5. O nosso except busca tratar o ValueError, como já fizemos inúmeras vezes neste tutorial. Só que agora usamos as para associar o ValueError capturado à variável “erro”!

Essa variável agora faz referência à exceção e podemos utilizá-la dentro do bloco except. Por exemplo, no nosso print() inserimos “erro” junto com o texto que escrevemos. Vamos causar uma exceção para ver o que acontece:



Tentamos passar “a” como valor, mas esse valor gerou um ValueError! Observe que a variável “erro” contém informações mais detalhadas: “invalid literal for int() with base 10: 'a'”.

Repare que esse é o mesmo texto que recebemos quando o Traceback aparece com aquela mensagem de erro! O que fizemos associar essas informações a variável para utilizarmos posteriormente em nosso programa!

ATENÇÃO: A variável que utilizamos para acessar as informações da exceção capturada somente deve ser utilizada dentro do bloco except. **Ao final desse bloco, o Python remove essa variável**, portanto não podemos utilizá-la posteriormente em outras partes do programa!

Por que utilizar as? O uso de as é útil quando precisamos obter **informações sobre a exceção que ocorreu**.

Quando uma exceção é tratada por um bloco except, o Python não exibe automaticamente o Traceback e a mensagem de erro que seriam apresentados caso a exceção não fosse tratada. Entretanto, podemos utilizar as para associar a exceção capturada a uma variável e, dessa forma, acessar informações relacionadas ao erro.

Essas informações podem ser utilizadas, por exemplo, para apresentar uma mensagem mais detalhada ao usuário, registrar o erro em um arquivo de log ou realizar algum outro tratamento específico.

4.7 Lançando Exceções – raise

A palavra reservada raise é utilizada para provocar (ou lançar) explicitamente uma exceção durante a execução de um programa. Até agora, vimos situações em que o próprio Python provoca uma exceção, porém, é possível que o programador determine explicitamente quando uma exceção deve ocorrer.

O raise tem a seguinte sintaxe:

raise TipoDaExcecao

raise TipoDaExecao(“mensagem”)

Vamos usar como exemplo um programa que pergunta a idade do usuário. Sabemos que não existe idade negativa. Caso o usuário digite um número negativo, vamos usar o raise para lançar um ValueError com uma mensagem explicando isso. Veja:

```
1   idade = int(input("Digite sua idade: "))
2
3   if idade < 0:
4       raise ValueError("A idade não pode ser negativa.")
5
6   print(f"Idade informada: {idade}")
7
```

```
Digite sua idade: -5
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\exceptions.py", line 4, in <module>
    raise ValueError("A idade não pode ser negativa.")
ValueError: A idade não pode ser negativa.

Process finished with exit code 1
```

Observe que, a princípio, -5 não seria problema nenhum para o nosso programa. Esse valor pode corretamente ser convertido para int na linha 1 do nosso código.

Entretanto, explicitamente determinamos que um valor negativo gerará uma exceção do tipo ValueError usando raise. Também passamos uma mensagem a ser exibida.

Com isso, forçamos o Python a lançar essa exceção e a exibir a mensagem que desejamos. Quando passamos o valor negativo, a exceção foi acionada e o programa foi interrompido, com a mensagem de erro e o Traceback.

Também, podemos tratar a exceção que levantamos com o raise dentro da estrutura try-except. Assim, podemos lançar a exceção sem que haja a interrupção do programa. Vamos continuar com o mesmo exemplo anterior:

```
try:
    idade = int(input("Digite sua idade: "))

    if idade < 0:
        raise ValueError("A idade não pode ser negativa.")

except ValueError as erro:
    print(f"Erro: {erro}")

```

```
Digite sua idade: -5
Erro: A idade não pode ser negativa.

Process finished with exit code 0
```

Como podemos ver, a nossa exceção agora não interrompeu o programa. Não foi gerada a mensagem de erro com o Traceback.

ATENÇÃO: Exceções não tratadas causam a interrupção do programa e o Python exibirá o Traceback. Isso também vale para as exceções lançadas com raise.

Por que utilizar raise? O raise é especialmente útil quando queremos estabelecer regras próprias para o nosso programa e informar que determinada situação não é válida.

Por exemplo, no nosso programa de idade, um valor negativo, por si só, não gera problema nenhum. O programador é que deseja que a idade não possa ser negativa, pois isso é algo que não existe no mundo real.

Porém, do ponto de vista do Python, -5 é um valor possível de ser convertido para int. Por isso é que se mostra razoável lançar uma exceção nesse caso: o Python aceita valores negativos nesse caso, mas o bom funcionamento do nosso programa não aceita essa situação!

ATENÇÃO: Enquanto algumas exceções são provocadas automaticamente pelo Python quando ocorre uma situação inesperada, o raise permite que o programador provoque explicitamente uma exceção quando identificar uma situação que deve ser considerada inválida pelo programa.

4.8 Considerações Finais

Com isso, finalizamos a Parte 3 desse tutorial. Tivemos um contato com as estruturas de dados e com o tratamento de exceções. Esses tópicos ainda serão aprofundados em momento oportuno.

Na próxima Parte, vamos continuar com as principais estruturas de dados – dicionários e conjuntos. Também teremos exercícios para fixar o conhecimento.